A PROJECT REPORT

ON

"PhantomIDS"

SUBMITTED TO THE MSBTE, MUMBAI
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD

OF

DIPLOMA IN COMPUTER TECHNOLOGY

By

Mr. Tanishq Shivasharan	**Enroll No: 23211020196**
Mr. Pavan Damji	**Enroll No: 23211020157**
Mr. Kaushik Hatte	**Enroll No: 23211020114**
Mr. Vedant Vallal	**Enroll No: 23211020117**

Under the Guidance of

Mrs. Trigule D.J.

Department of Computer Technology

Solapur Education Society's Polytechnic, Solapur.

2025 - 2026

CERTIFICATE

This is to certify that the project report entitled

"PhantomIDS"

Submitted by:

Mr. Tanishq Shivasharan	**Exam Seat. No: 184378**
Mr. Pavan Damji	**Exam Seat. No: 184342**
Mr. Kaushik Hatte	**Exam Seat. No: 184302**
Mr. Vedant Vallal	**Exam Seat. No: 184305**

is a bonafide work carried out by them under the guidance of **Mrs. Trigule D.J.** and it is approved for the partial fulfillment of the requirement of MSBTE, MUMBAI for the award of the Diploma in Computer Technology

This project work has not been earlier submitted to any other Institute or University for the award of any degree or diploma.

Mrs. Trigule D.J.	**Mr. M. C. Patil**	**Mr. A. A. Bhawtankar**	**External**
Project guide	**Head** Department Of Computer Technology	**Principal** S. E. S. Polytechnic, Solapur	**Examiner**

Place: Solapur.

Date: / /2026

DECLARATION

I hereby declare that this project work entitled **"Phantom IDS"** has been prepared by me during the academic year 2025–26 under the guidance of **Mrs. Trigule D.J.,** Department of Computer Technology, S. E. S. Polytechnic, Solapur in the partial fulfillment of diploma degree prescribed by the college.

I also declare that this project is the outcome of my own effort, that it has not been submitted to any other university for the award of any degree.

Student Name & Signature

Mr. Tanishq Shivasharan :-

Mr. Pavan Damji :-

Mr. Kaushik Hatte :-

Mr. Vedant Vallal :-

ACKNOWLEDGEMENT

It gives us a great pleasure to present the project report entitled **PhantomIDS.**

This project has given us a great learning experience in understanding the various aspects of the Android app designing. So, we would like to acknowledge all the minds and hands that contributed in giving this project a proper shape.

We are grateful to our ***Hon. Guide Mrs. Trigule D. J.*** our Project Guide for her encouragement and guidance & keen interest throughout project. We are also thankful to our ***Head of Department Mr. M. C. Patil*** and our ***Principal A. A. Bhawtankar*** for their moral support.

We are thankful to all the staff and non-teaching members of Computer Technology Department for their co-operation during the project work. We are very grateful to those who in the form of books had conveyed guidance in this project work.

ABSTRACT

SQL Injection (SQLi) remains one of the most pervasive and damaging web security threats, frequently exploited to bypass authentication and exfiltrate sensitive data. Traditional software-based **Web Application Firewalls (WAFs)** and **Intrusion Detection Systems (IDS)** are typically resident on the same host they protect, leaving them vulnerable to being disabled if an attacker gains root or shell access to the server. This study introduces **PhantomIDS**, a cyber-physical security framework that addresses this limitation by decoupling the detection sensor from the target host.

The **methodology** employs a multi-layered, hybrid defense strategy combining a physical hardware sensor, a deliberately vulnerable honeypot, and a machine learning (ML) backend. At the network layer, an **ESP32 microcontroller** operates as a **transparent HTTP proxy** within a WiFi broadcast domain. It intercepts traffic destined for the "CorpNet Employee Portal"—a Flask-based honeypot mimicking a corporate login—to perform real-time inspection. The hardware layer utilizes a **two-signal confidence scoring mechanism**, cross-referencing URL-decoded payloads for suspicious characters (quotes) and a 20-entry SQL keyword list.

Detections are further validated by a Node.js backend using a **Logistic Regression model** that analyzes behavioral features such as request rates and threat counts. Experimental results demonstrate that this hybrid approach achieves an **86.14% detection accuracy** and an **87.93% F1 score**, specifically identifying the characteristic rapid sequential probes of automated tools like sqlmap. Automated mitigation is handled through a deterministic **3-strike auto-ban engine**, which permanently blocks malicious IPs. By utilizing **physically isolated hardware**, PhantomIDS ensures that security monitoring remains independent and resilient even if the primary web server is compromised, providing a robust platform for real-time intrusion detection and forensic analysis.

Keywords: SQL Injection, ESP32, Hardware IDS, Machine Learning, Honeypot, Network Security, Transparent Proxy.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION (BACKGROUND OF PROBLEM)

1. INTRODUCTION

PhantomIDS is a cyber-physical security framework designed to resolve the "host-dependency" flaw inherent in traditional software firewalls. While industry-standard software WAFs are vulnerable to being disabled if an attacker gains shell access, **PhantomIDS** utilizes a physically separate **ESP32 hardware sensor** to sniff network traffic out-of-band, making it **immune to software-level compromise**.

By pairing a vulnerable decoy honeypot with a **Hybrid ML Anomaly Detection pipeline**—leveraging LSTM behavioural models—the system identifies zero-day SQLi variants through mechanical timing and payload entropy rather than static signature matching. This architecture provides a resilient, un-killable defence layer that continues to alert via its **physical buzzer and LCD** even when the host operating system is fully compromised.

1.1. BACKGROUND

SQL Injection (SQLi) remains a pervasive network threat, enabling attackers to bypass authentication and exfiltrate sensitive data using automated tools like sqlmap. To combat these exploits, major tech providers like Amazon, Google, and Microsoft rely on Layer 7 Web Application Firewalls (WAFs)—such as AWS WAF and Azure WAF—which inspect HTTP traffic using signature-based matching and rule sets like the OWASP Core Rule Set. While effective against known patterns, these software-defined filters are often bypassed by novel encoding or high-interaction attacks.

The fundamental limitation of traditional software security is **host dependency**. Because software WAFs and IDS run as modules or agents within the same operating system they protect, they share a critical single point of failure: if an attacker achieves shell or root access, they can simply **disable or delete the security process**. Additionally, software parsers often suffer from "conformance mismatches," where techniques such as wrapping SQL payloads in JSON syntax can blind the inspection engine to the malicious activity.

To resolve these architectural flaws, **PhantomIDS** introduces a "Hardware-First" thesis using a physically separate ESP32 microcontroller as an out-of-band sensor. By sniffing traffic at the network layer independently of the host, the system remains **"un-killable" by software-level compromise**. This hardware resilience is paired with a transition to behavioral **Machine Learning anomaly detection**—utilizing models like LSTM and Isolation Forests—to identify the mechanical "rhythm" of automated tools and catch zero-day SQLi variants that static software rules frequently miss.

1.2.PROBLEM STATEMENT

Traditional Web Application Firewalls (WAFs) and Intrusion Detection Systems (IDS) are typically software-based and resident on the same host they are designed to protect. This creates a critical vulnerability: if an attacker gains root or shell access to the server, they can easily disable or delete the security process. Furthermore, standard signature-based filters often fail to detect obfuscated or zero-day SQL injection (SQLi) attacks and the rapid sequential probing patterns of automated tools like sqlmap.

1.3.PROJECT OBJECTIVES

- Physical Isolation: Develop a standalone hardware IDS using an ESP32 microcontroller that monitors traffic independently of the target host.
- Real-Time Inspection: Act as a transparent HTTP proxy to intercept, URL-decode, and inspect raw request bodies for malicious payloads.
- Hybrid Detection: Combine signature-based keyword matching at the hardware layer with a Logistic Regression ML model on the backend to achieve high detection accuracy.
- Forensic Data Collection: Deploy a vulnerable honeypot to lure attackers and capture their characteristic probe patterns for analysis.
- Automated Mitigation: Implement a 3-strike auto-ban engine and physical alerting (LCD, buzzer, LED) to immediately neutralize and report network threats.

1.4. PROJECT PLAN

Planned Date	Planned Work
September 1st week	Studying activities in capstone project planning and understanding IoT-based security system concepts
September 2nd week	Decide criteria for group members & group formation based on skills (hardware, software, networking)
September 3rd week	Finalization of group members for PhantomIDS project
September 4th week	Research on SQL injection attacks, IDS systems, ESP32 capabilities, and existing honeypot solutions
October 1st week	Finalization of project title "PhantomIDS", problem definition — hardware IDS gap in SMB security
October 2nd week	Prepare survey questions on SQL injection awareness and current IDS usage in organizations
October 3rd & 4th week	Research on existing papers on network-layer IDS, honeypot design, and ESP32 Wi-Fi sniffing techniques
November 1st week	Preparation of system architecture, data flow diagram of attack-detect-alert pipeline, and industrial survey
December 1st , 2nd week	Preparing project report outline — introduction, problem statement, objectives, and literature review
December 3rd , 4th week	Study of required tools — Arduino IDE, Node.js, SQLite, SQL map, Wireshark, LiquidCrystal_I2C library
January 1st , 2nd week	Design basic system structure — three-actor model (Kali attacker, Express honeypot, ESP32 IDS sensor)
January 3rd , 4th week to February 1st week	Design and implement honeypot Flask web application with vulnerable login form, SQLite DB, and attack logging
February 2nd week to March 1st week	Implement ESP32 IDS sketch — TCP server, URL decoder, 2-signal detection engine, LCD + buzzer + LED alerts

March 2rd week	Testing individual modules — honeypot login, sqlmap attack simulation, ESP32 WiFi connection and TCP reach
March 3rd week	Integrating all modules — ESP32 intercepts traffic, detects SQLi, forwards to honeypot, fires real-time alert
March 4th week	Test the complete integrated system — run full sqlmap attack, verify ESP32 detects and logs all injections
April 1st week	Deploy and test PhantomIDS on local WiFi network with Kali VM (bridged adapter), validate end-to-end flow
April 1st week	Testing the complete system on LAN with all three actors — Kali, Windows honeypot, ESP32 sensor
April 2nd week	Taking approval from guide. Prepare final PhantomIDS CPE Project Report

Table 1.2.B.1: Project Plan

CHAPTER 2: LITERATURE SURVEY

2.1. EVOLUTION OF NETWORK SECURITY AND IDS

The landscape of network security has evolved from rudimentary 1980s packet filtering to sophisticated Intrusion Detection and Prevention Systems (IDS/IPS) in the 1990s. Early systems relied heavily on signature-based detection, which matches byte sequences in payloads against a database of known threats. While accurate for known patterns, these systems struggle with zero-day attacks. Modern paradigms like Secure Access Service Edge (SASE) and Zero-Trust Network Access (ZTNA) have shifted security to the cloud, yet the core challenge of distinguishing malicious traffic from legitimate user behavior remains.

2.2. CURRENT INDUSTRY STANDARDS: CLOUD-BASED WAFS

Enterprise-grade solutions such as AWS WAF, Azure WAF, and Google Cloud Armor primarily protect against OWASP Top 10 threats, including SQLi. These platforms utilize Core Rule Sets (CRS) to block common attack patterns. For example, AWS WAF employs Security Automations and Bad Bot custom rules to deflect automated intrusions. However, research by Claroty's Team82 revealed that major WAFs (including AWS and F5) could be bypassed by prepending JSON syntax to SQLi payloads, as WAF parsers often lacked the depth to recognize these mismatches while the backend database engine executed them.

2.3. MACHINE LEARNING FOR PREDICTIVE SQLI DETECTION

Recent research has shifted toward **machine learning (ML)** to overcome the static nature of rule-based filters.

- Predictive Frameworks: Nandigama (2024) demonstrated that cloud-native ML services (like AWS SageMaker) using LSTM models can capture semantic query structures and detect zero-day SQLi variants with an F1-score of 0.94.
- Dual-Layer Architectures: Sameh and Selim (2024) proposed an Adaptive Dual-Layer WAF (ADL-WAF). Their methodology uses a Decision Tree (DT) for initial anomaly detection and a Support Vector Machine (SVM) to classify threats, achieving 99.88% accuracy and reducing false positives to zero in experimental settings.
- Intelligent Detection: Industry players like NSFOCUS have integrated advanced semantic analysis and ML, reporting a SQLi false negative rate as low as 0.026%, far superior to traditional signature models.

2.4. HARDWARE ISOLATION AND THE "PHANTOMIDS" GAP

A critical limitation identified in host-based security systems (e.g., ModSecurity) is that they run as modules within the web server process; if an attacker gains shell access, they can simply disable the security layer. The NIST SP 800-94 guide provides foundational principles for IDS, but PhantomIDS addresses the "host compromise" gap by utilizing physical isolation. By deploying an ESP32 microcontroller as a transparent HTTP proxy, the monitoring sensor remains out-of-band and unreachable via software-level attacks (e.g., "rm -rf"), ensuring persistent network oversight even if the target machine is compromised.

This project bridges the gap between the high-accuracy behavioral analysis seen in ADL-WAF research and the resilient hardware sensors required for robust industrial defense.

CHAPTER 3: SCOPE OF THE PROJECT

3. SCOPE OF THE PROJECT

The PhantomIDS project is designed as a cyber-physical security framework specifically focused on the detection, logging, and mitigation of SQL Injection (SQLi) attacks within a local network environment. The scope of the project encompasses hardware sensing, behavioral analysis through machine learning, and forensic data collection via a honeypot.

3.1 FUNCTIONAL REQUIREMENTS

The system is engineered to provide a multi-layered defense mechanism:

- Real-Time Traffic Inspection: Utilizing an ESP32 microcontroller as a transparent HTTP proxy to intercept and URL-decode raw POST request bodies.
- Signature-Based Detection: Implementing a two-signal confidence scoring mechanism that matches suspicious characters (quotes) and a 20-entry SQL keyword list (e.g., UNION, SELECT, DROP).
- Behavioral Anomaly Detection: Leveraging a Logistic Regression ML model to analyze request rates and cumulative threat counts, achieving a documented accuracy of 86.14%.
- Automated Mitigation: Executing a deterministic 3-strike auto-ban engine that blocks malicious IP addresses at the application layer.
- Physical Alerting: Providing immediate out-of-band feedback through hardware components including a 16x2 LCD display, a red LED, and an active buzzer.

3.2 HARDWARE & SOFTWARE SPECIFICATIONS

The project operates within a bounded technical environment to ensure high-fidelity monitoring:

- Network Topology: The system is designed for a WiFi broadcast domain where all actors (attacker, sensor, and target) reside on the same subnet (e.g., 192.168.1.x).
- Protocol Support: The current scope is limited to HTTP traffic. HTTPS/TLS traffic is explicitly excluded as wire-level inspection is prevented by encryption.
- Hardware Architecture: Uses the FreeRTOS dual-core capabilities of the ESP32 to separate packet capture (Core 0) from detection logic (Core 1), preventing packet loss during high-volume scans from tools like sqlmap.

3.3 PROJECT CONSTRAINTS AND LIMITATIONS

To maintain focus on the core research thesis, the following boundaries are established:

- Attack Vectors: The project focuses on first-order SQLi. Second-order injections and other vulnerabilities like XSS or RCE are documented as future extensions.
- Environment: PhantomIDS is built for educational and research purposes in controlled lab settings. It is not intended for deployment on public-facing production systems.

- Platform Isolation: A primary scope objective is physical isolation; the IDS remains independent of the target host so that it cannot be disabled even if the web server process is compromised.

CHAPTER 4: METHODOLOGY

4.1. OVERALL SYSTEM ARCHITECTURE

The methodology for PhantomIDS is centered on a cyber-physical framework comprising three primary actors: an attacker utilizing Kali Linux and sqlmap, a deliberately vulnerable honeypot representing a corporate portal, and a physically isolated ESP32-based hardware sensor. Unlike host-based security systems that share resources with the protected application, this architecture uses the ESP32 as a transparent HTTP proxy that intercepts and inspects traffic at the network layer independently of the target machine. This out-of-band monitoring ensures that even if the web server process is compromised, the detection system remains unreachable and functional.

4.2 NETWORK TOPOLOGY AND COMMUNICATION

The system operates within a WiFi broadcast domain where all devices share a common subnet (192.168.1.x). The attacker's environment is hosted in a VirtualBox VM with Bridged Adapter mode to ensure it maintains a unique LAN identity, making its packets visible to the ESP32 sensor. The ESP32 is configured in WiFi station mode, listening for traffic destined for the honeypot IP (192.168.1.6) on port 5000. When a request is intercepted, the hardware analyzes the raw HTTP payload and then relays the request to the honeypot backend; if the backend is unreachable, the sensor returns a 502 Bad Gateway response.

4.3 HARDWARE SENSOR DESIGN AND DETECTION LOGIC

The physical sensor utilizes an ESP32 DevKit V1, a 16x2 I2C LCD display, and an active buzzer. The firmware employs a two-signal confidence scoring mechanism for real-time detection. Signal 1: Quote Detection — The system URL-decodes the POST request body and checks for the presence of single (') or double (") quotes. Signal 2: Keyword Matching — The payload is cross-referenced against a 20-entry list of SQL keywords, including UNION, SELECT, DROP, and common tautologies like 1=1. The scoring logic triggers a Critical Alert (red LED, triple buzzer burst, and LCD notification) only when both signals are detected simultaneously, thereby reducing false positives caused by legitimate character usage.

4.4 BACKEND AND MACHINE LEARNING PIPELINE

The backend is developed using Node.js and Express, hosting the honeypot application and an ML-driven threat detector. The honeypot uses an intentionally unsanitized SQL query with direct string interpolation to facilitate the recording of successful injection probes. To handle the high request volume generated by tools like sqlmap, the SQLite3 database is configured with Write-Ahead Logging (WAL) mode, which enables concurrent read/write operations without locking errors.

Logistic Regression Model (Model 1) The first layer of backend detection uses a Logistic Regression algorithm that analyzes two primary behavioral features: request_rate (requests per minute) and threat_count (accumulated strikes). The model uses min-max scaling for normalization and a sigmoid activation function to determine the probability of a threat. This model currently operates with a documented accuracy of 86.14%.

3-Strike Auto-Ban Engine (Model 2) The second layer is a deterministic mitigation engine. Each time Model 1 identifies a request as a threat, the source IP receives one strike. Upon reaching

three strikes, the engine permanently bans the IP address, effectively neutralizing automated scanning tools while maintaining a zero false-ban rate for legitimate users.

4.5 IMPLEMENTATION PHASES

The project implementation follows a structured six-phase plan:

1. Environment Setup: Preparing the development environment and tools.
2. Network Setup: Configuring the broadcast domain and static IP addressing.
3. Honeypot Build: Developing the vulnerable portal and logging systems.
4. Hardware Test: Verifying I2C communication, LCD output, and TCP reachability.
5. IDS Logic: Implementing signature-based detection and FreeRTOS dual-core processing to separate packet capture from logic.
6. Integration: Final testing and deployment of the hybrid ML defense pipeline.

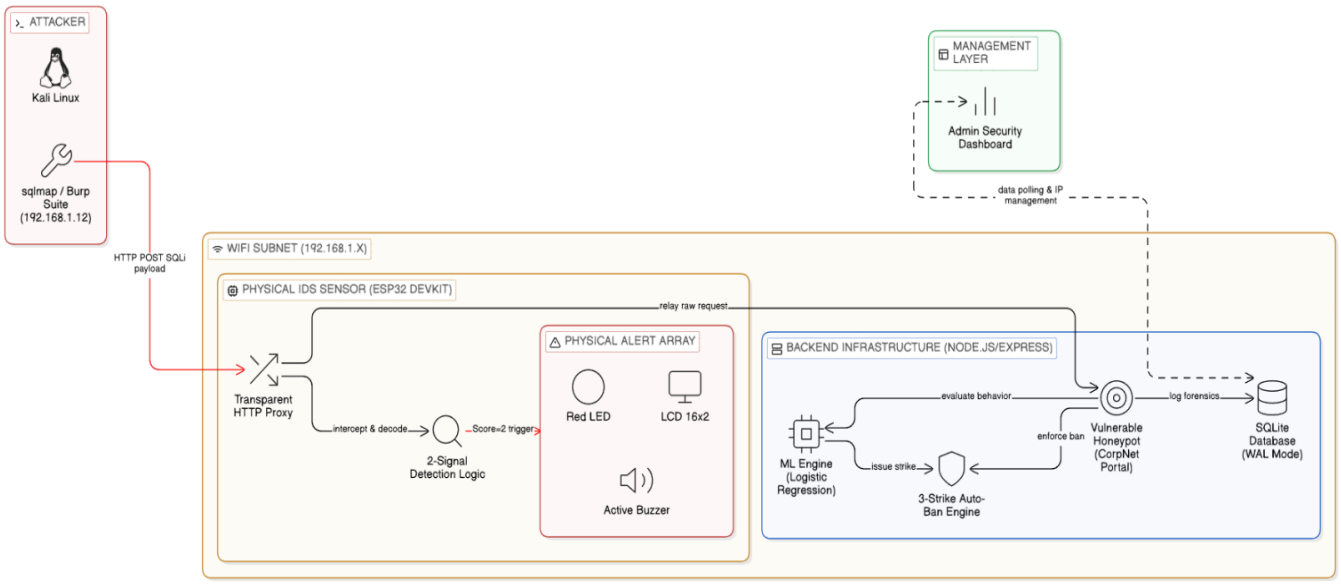

Figure 4 Architecture and Workflow diagram

CHAPTER 5: DESIGN, WORKING AND PROCESSES

5.1. UML DIAGRAMS

5.1.1 DFD Diagrams

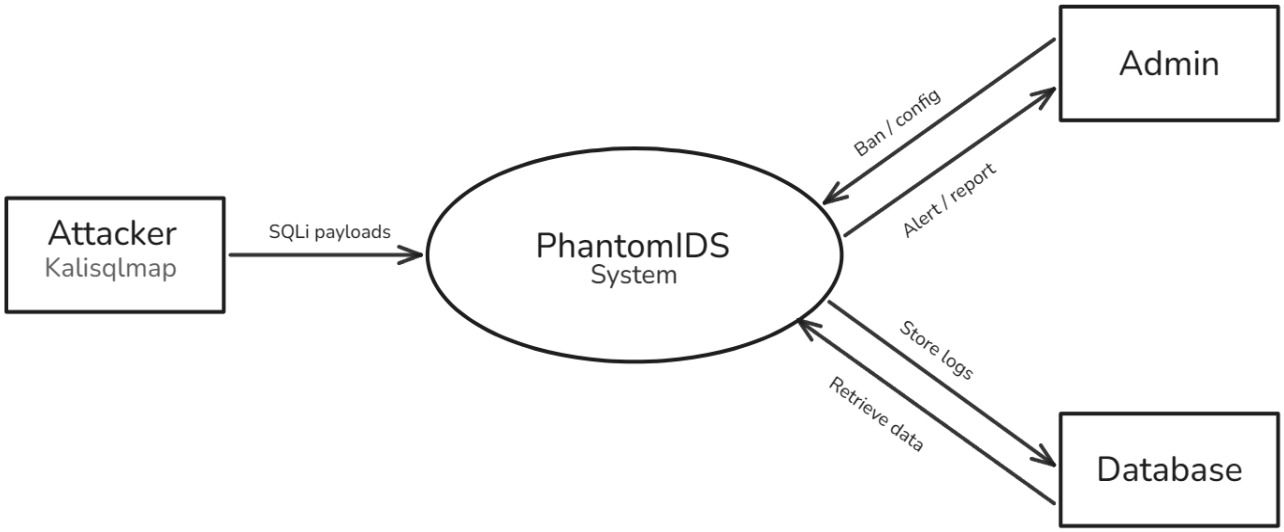

Figure 5.1.1.1 Level 0 DFD

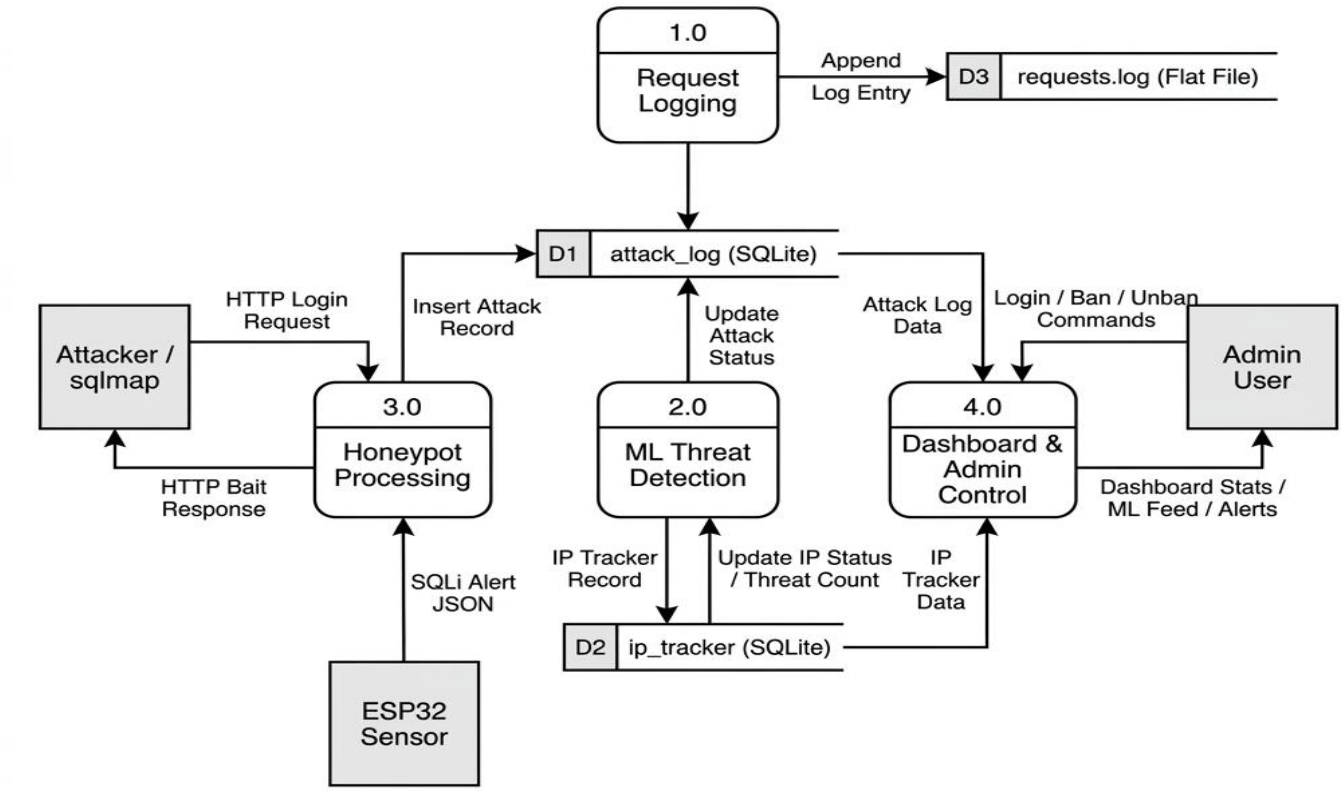

Figure 5.1.1.2 Level 1 DFD

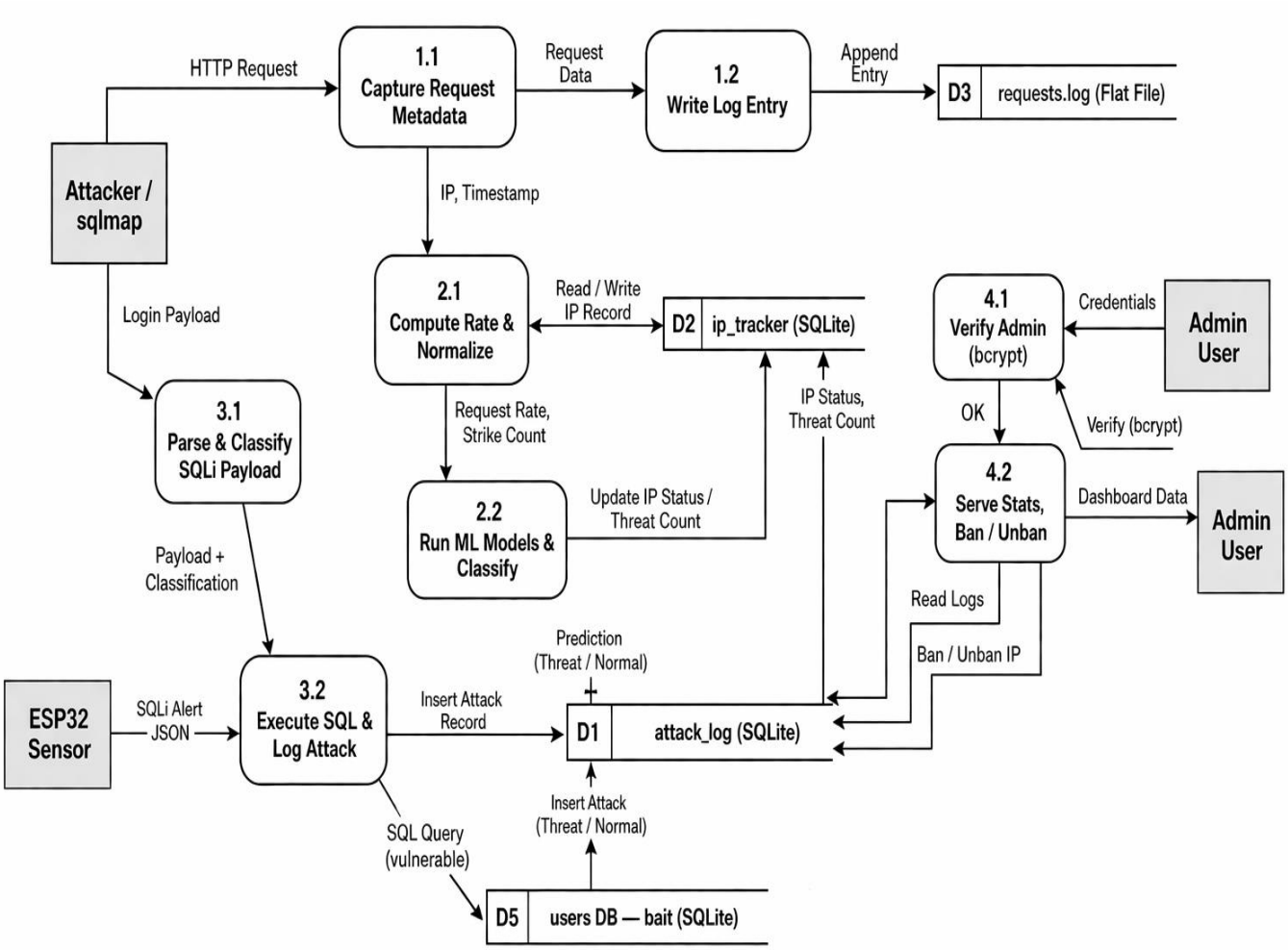

Figure 5.1.1.3 Level 2 DFD

5.1.2 CLASS DIAGRAM

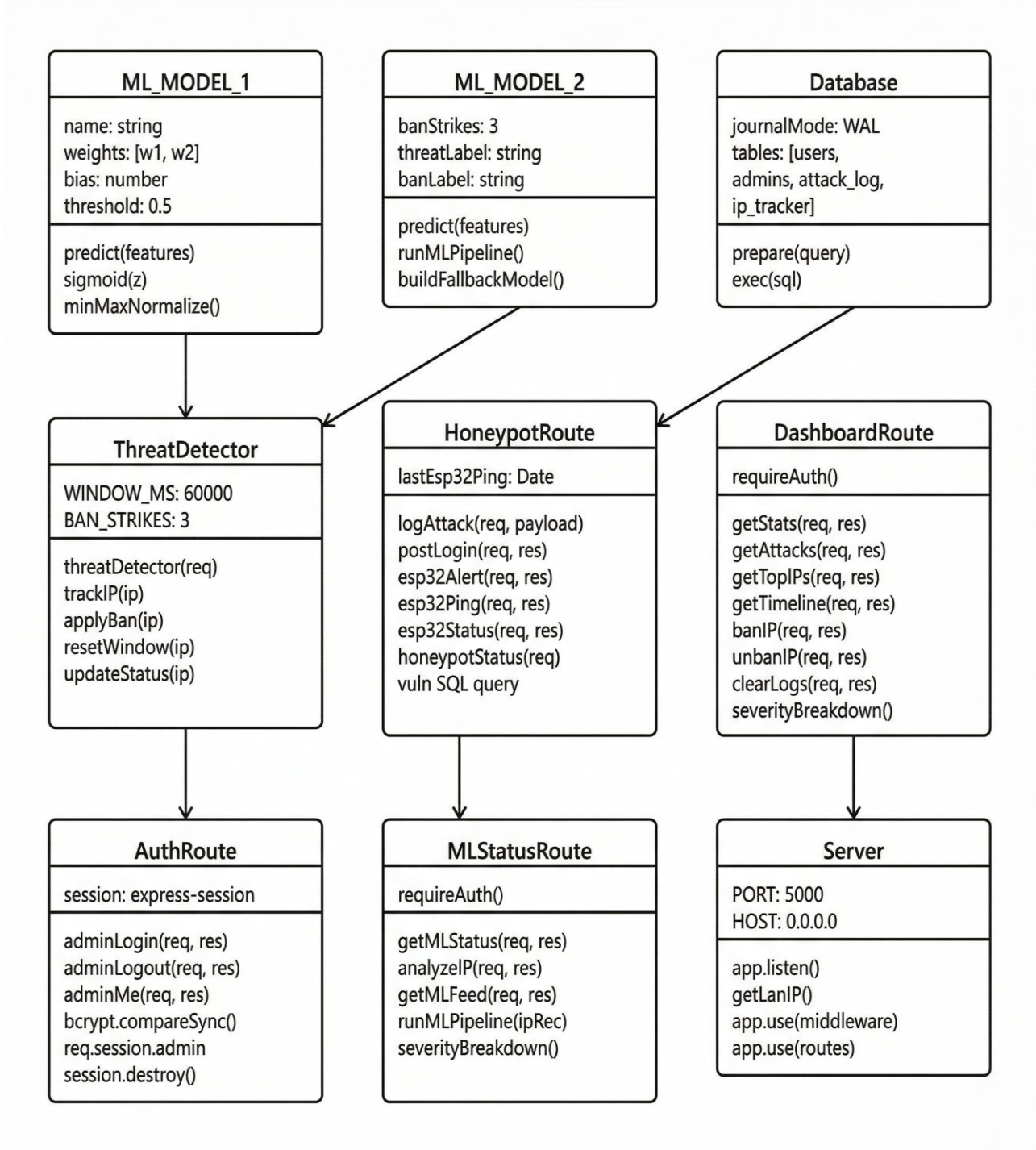

Figure 5.1.2 : Class Diagram

5.1.3 USE-CASE DIAGRAM

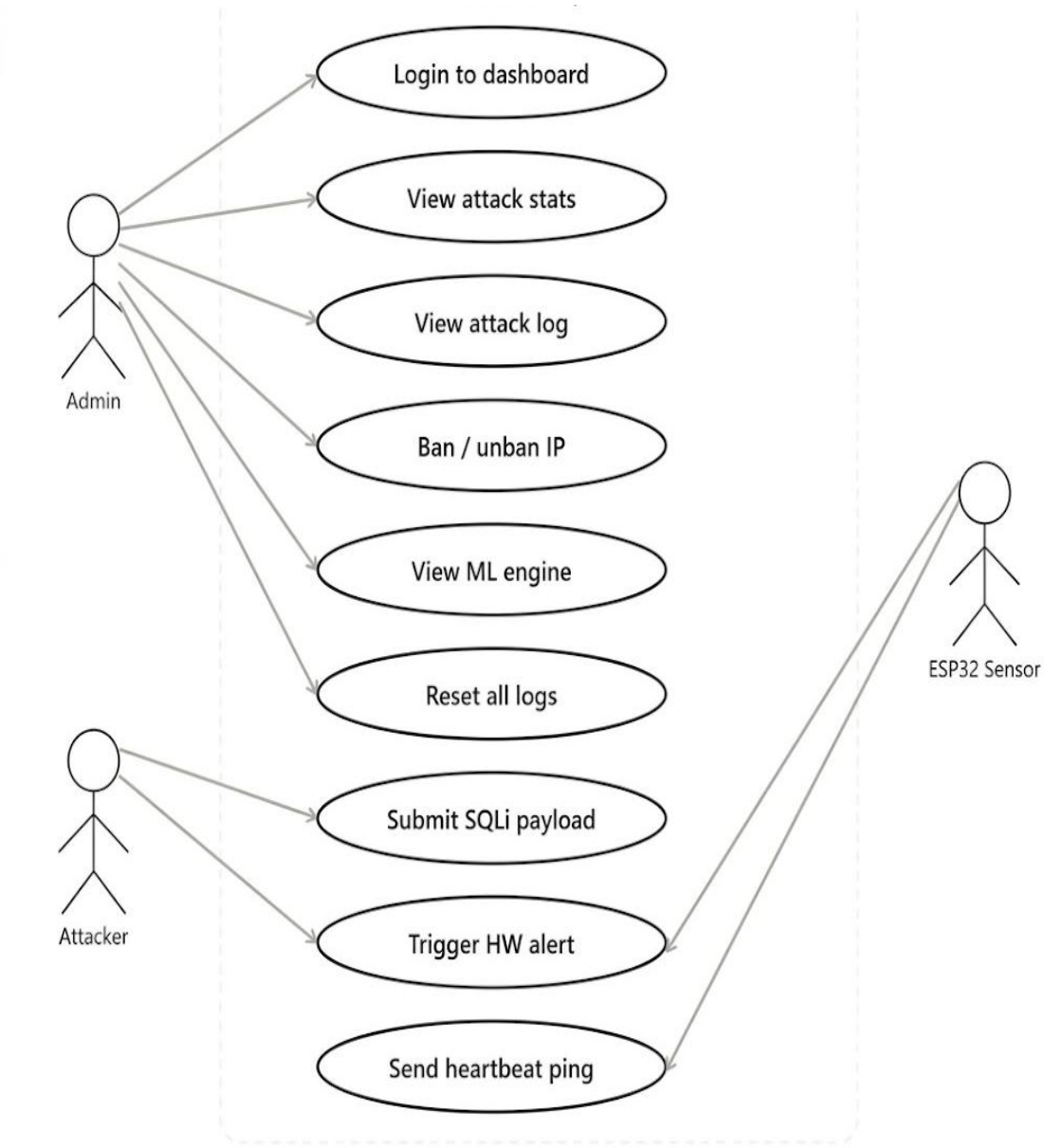

Figure 5.1.3 : Use Case Diagram

5.2 PROJECT SCREENSHOTS

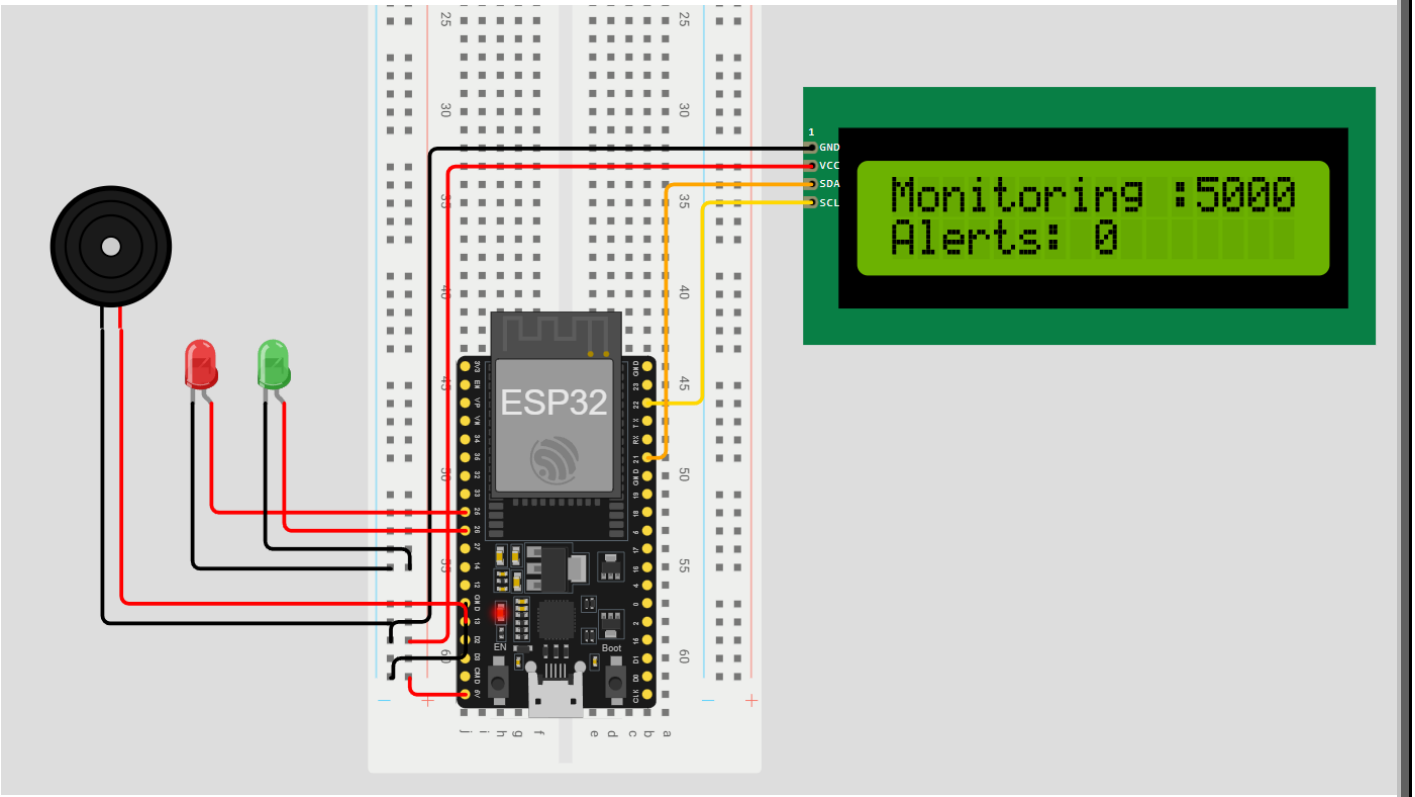

FIGURE 5.2.1 HARDWARE CONFIGURATION

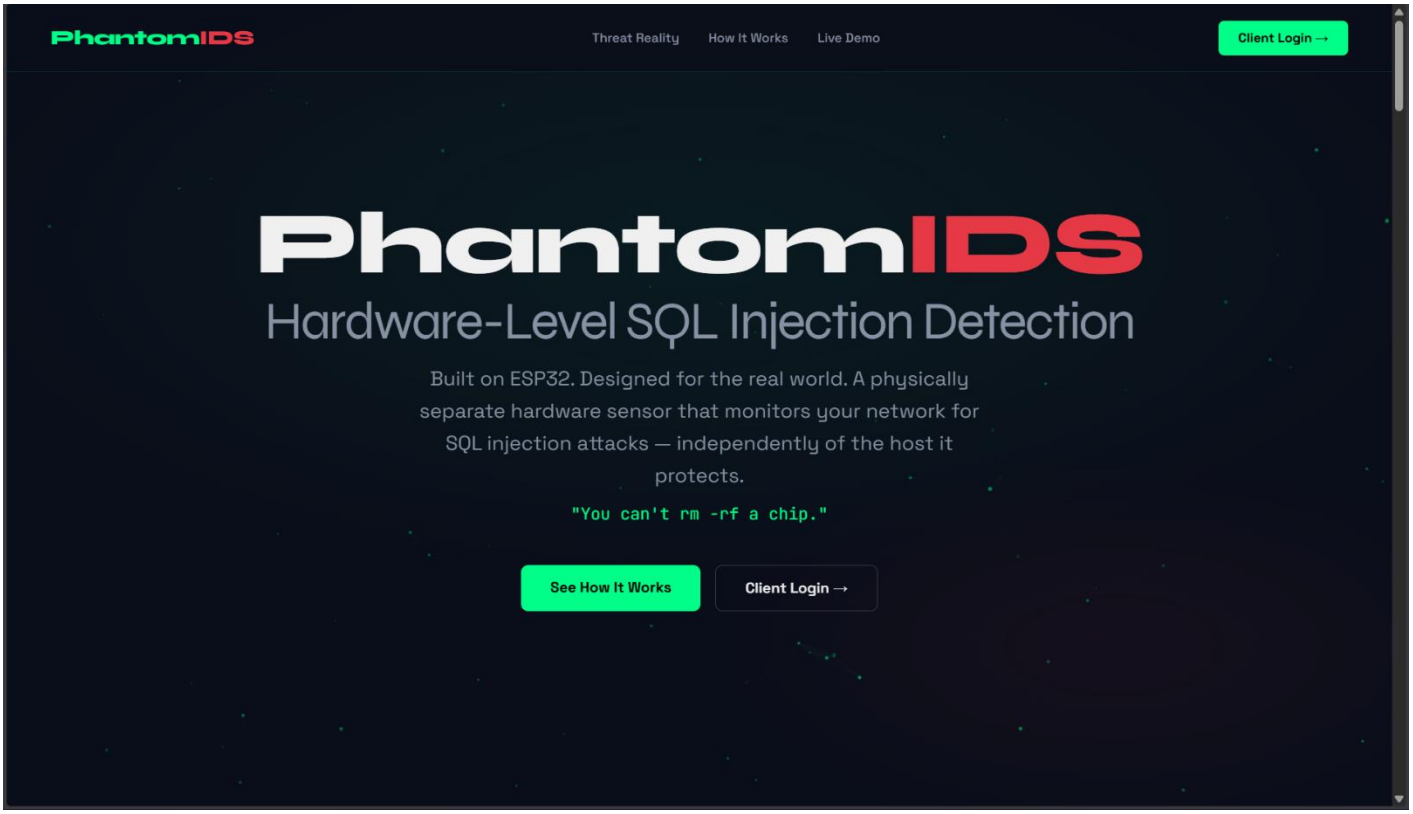

FIGURE 5.2.2 LANDING PAGE

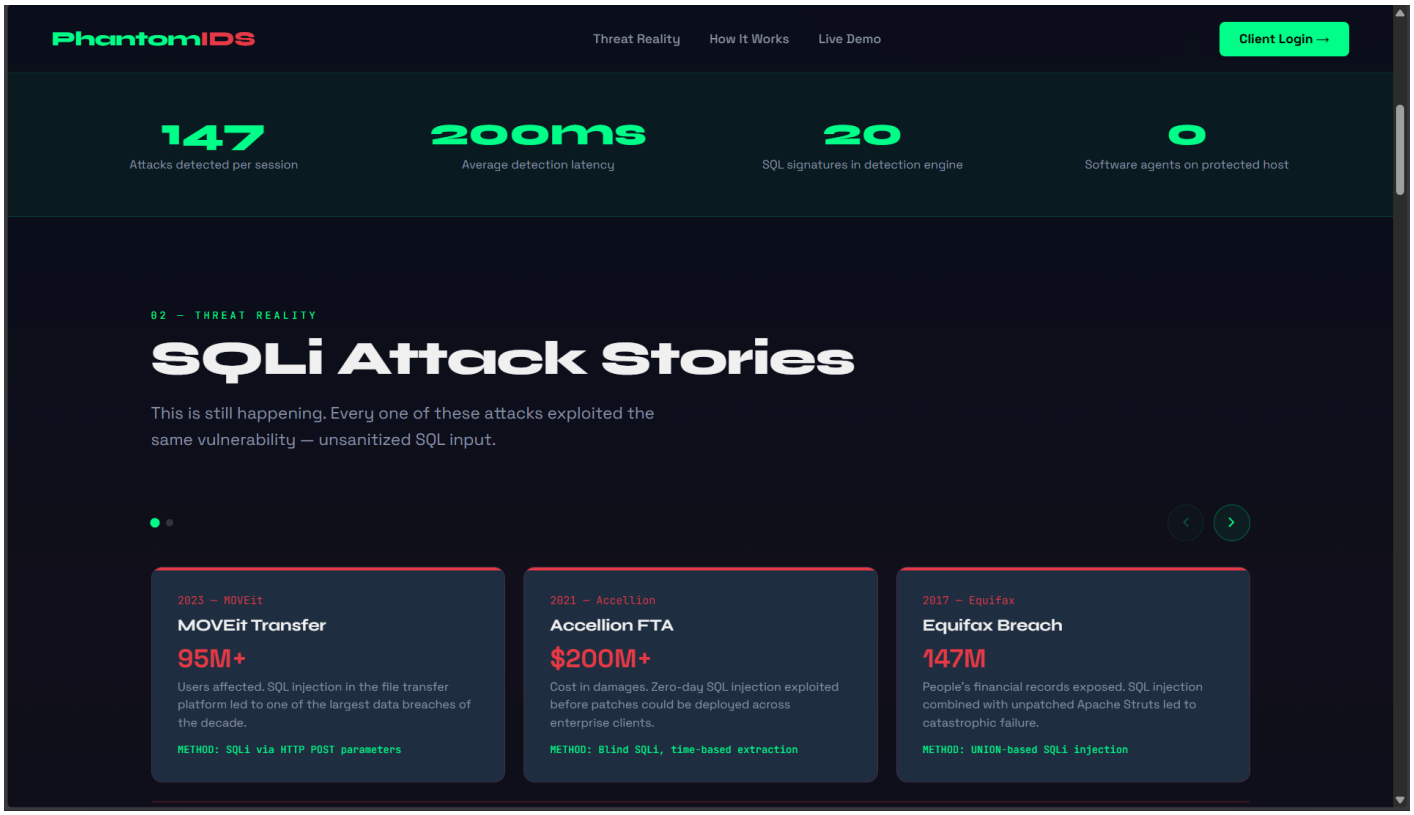

FIGURE 5.2.3 LANDING PAGE (PERFORMACE MATRIX & STORIES)

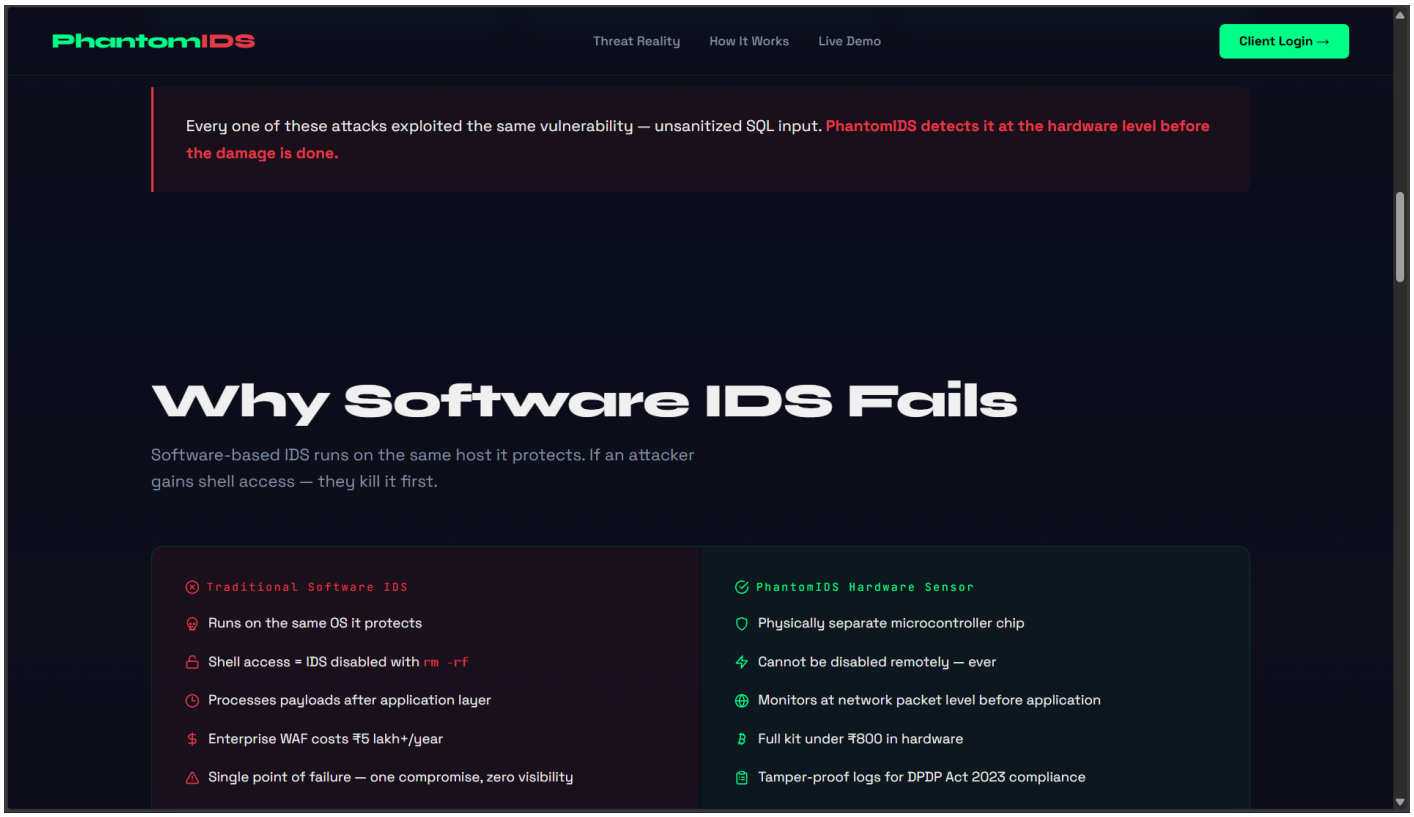

FIGURE 5.2.4 LANDING PAGE (DIFFERENCE)

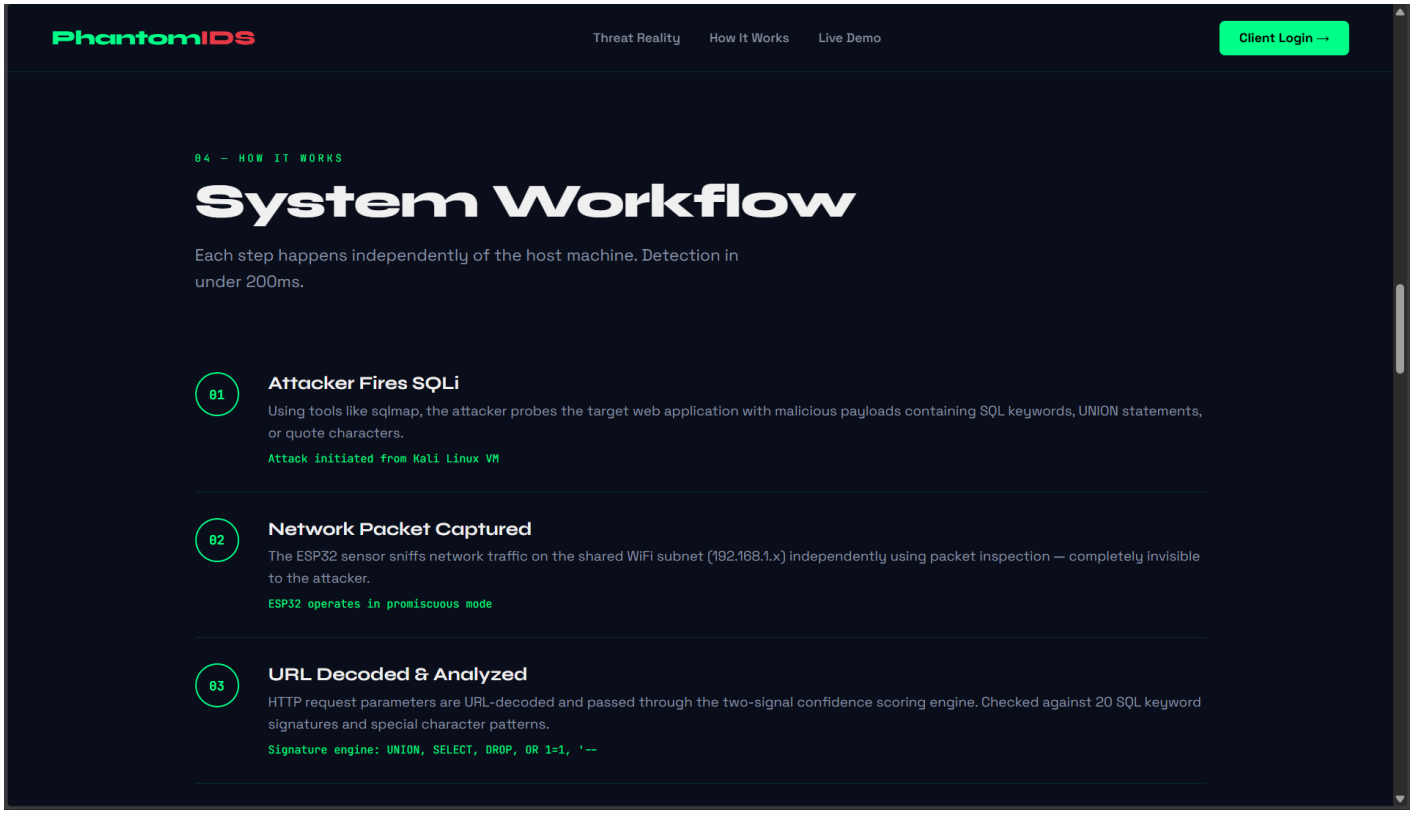

FIGURE 5.2.5 LANDING PAGE (SYSTEM WORKFLOW)

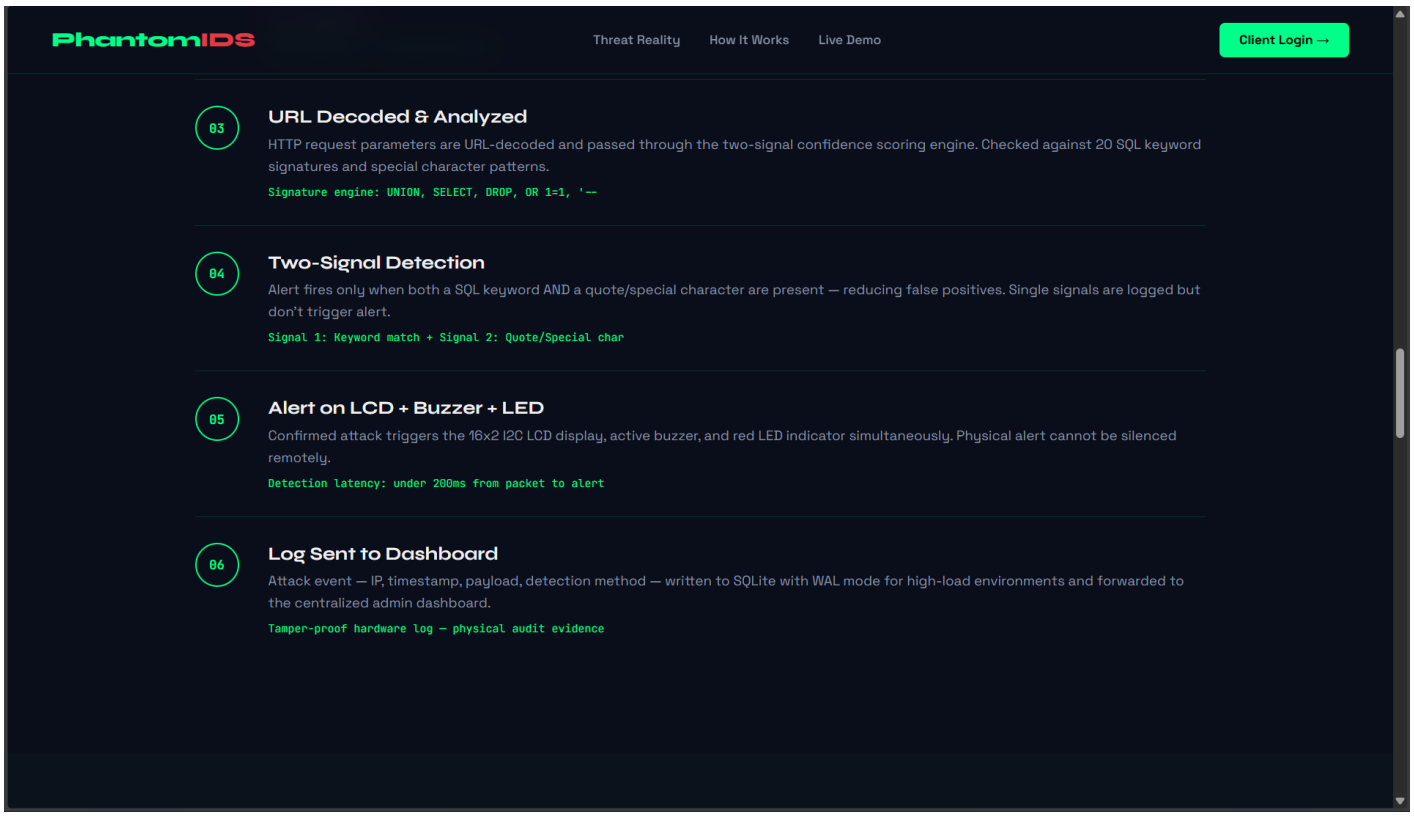

FIGURE 5.2.6 LANDING PAGE (SYSTEM WORKFLOW CONTINUED)

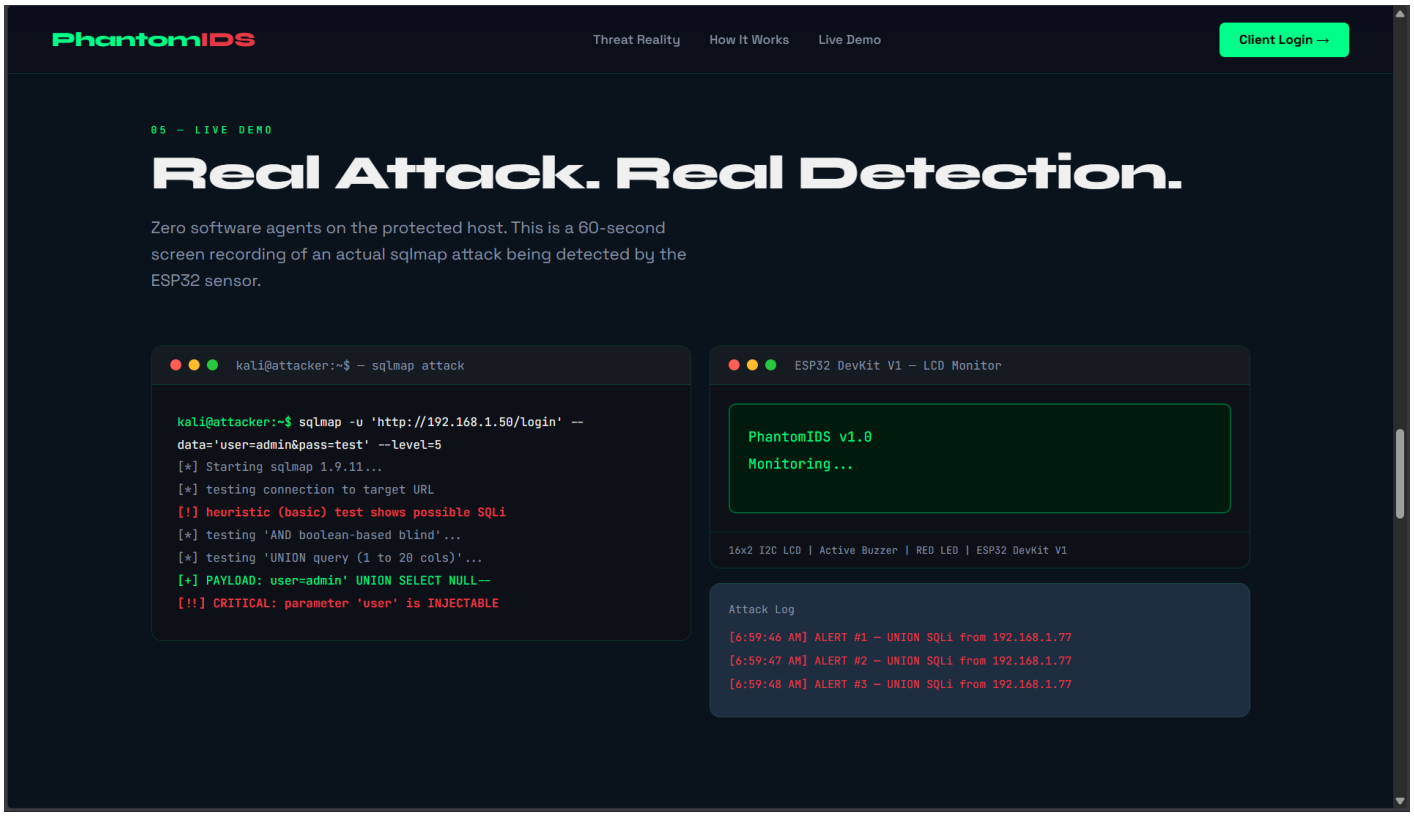

FIGURE 5.2.7 LANDING PAGE (SIMULATION MONITORING [TYPING SVG])

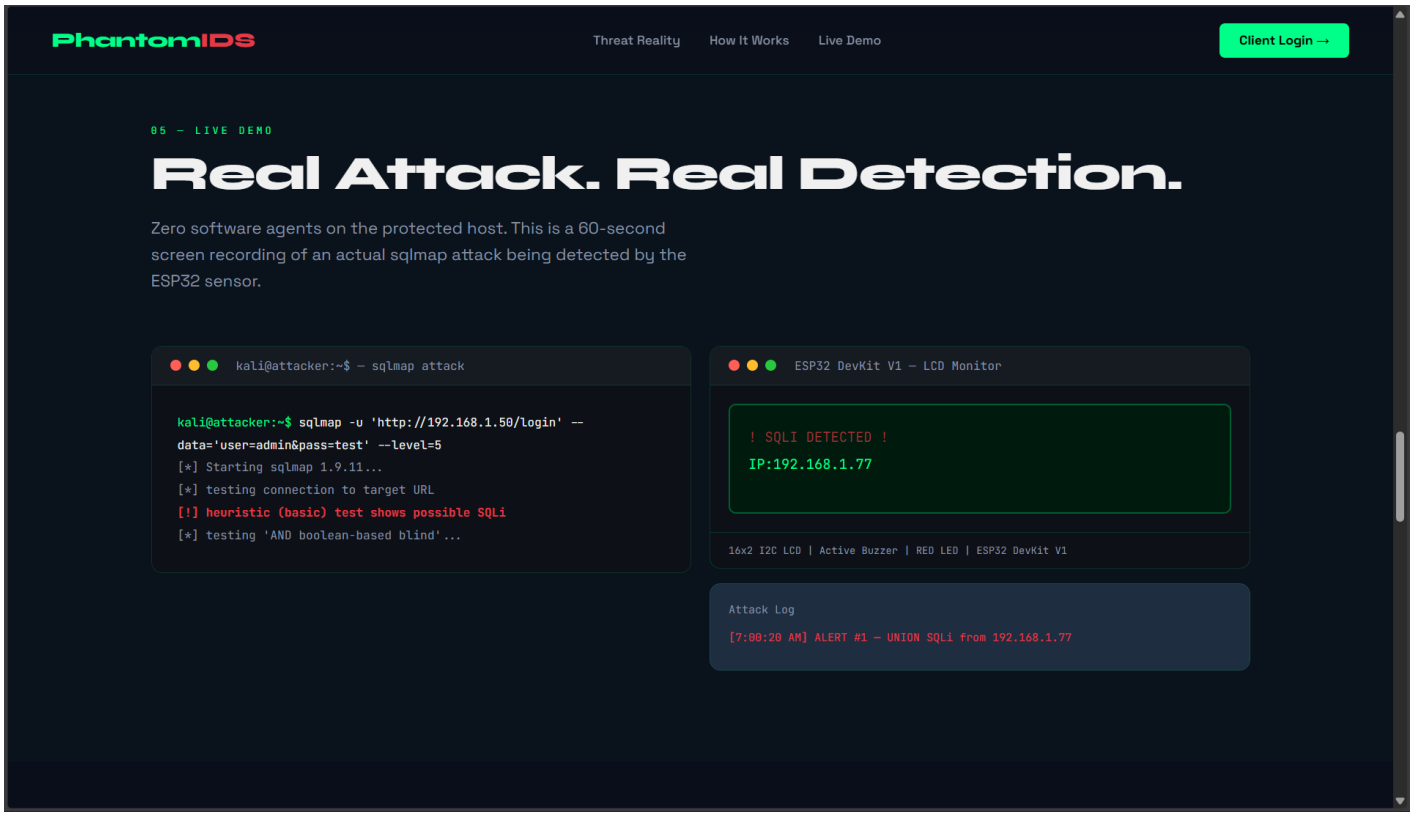

FIGURE 5.2.8 LANDING PAGE (SIMULATION DETECTED [TYPING SVG])

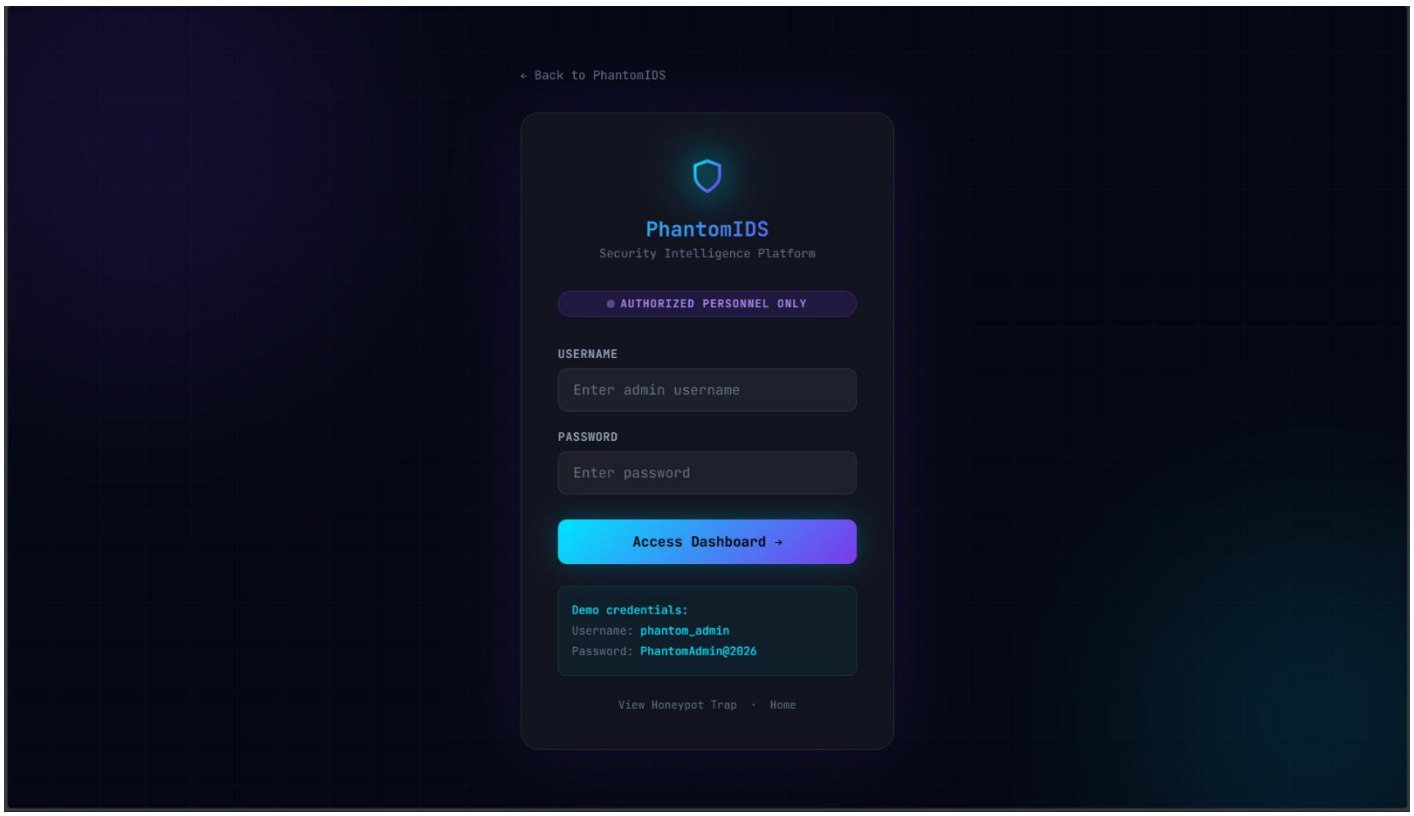

FIGURE 5.2.9 LOGIN PAGE (DEMONSTRATION LOGIN PAGE)

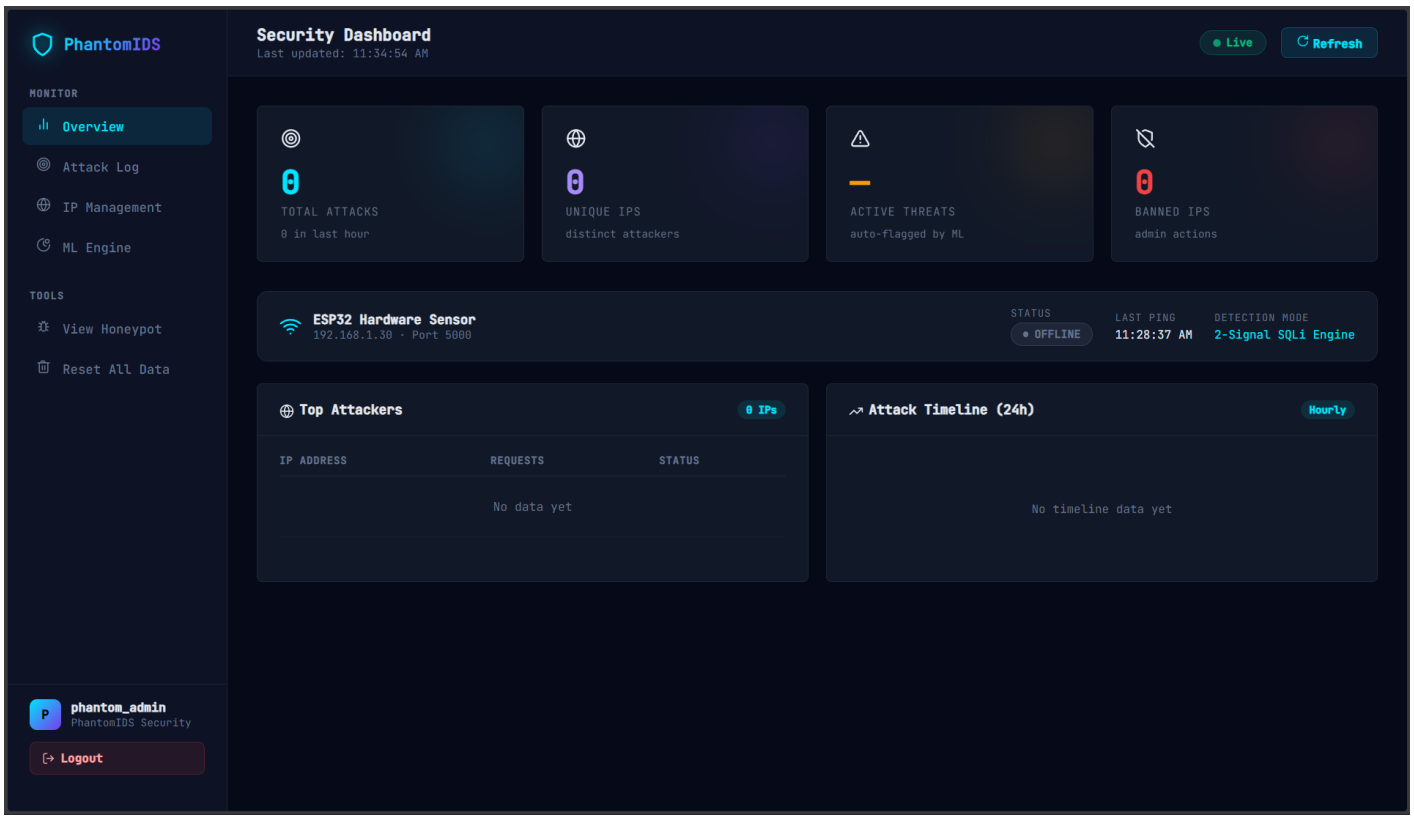

FIGURE 5.2.10 ADMIN DASHBOARD OVERVIEW SECTION (INITIAL STATE)

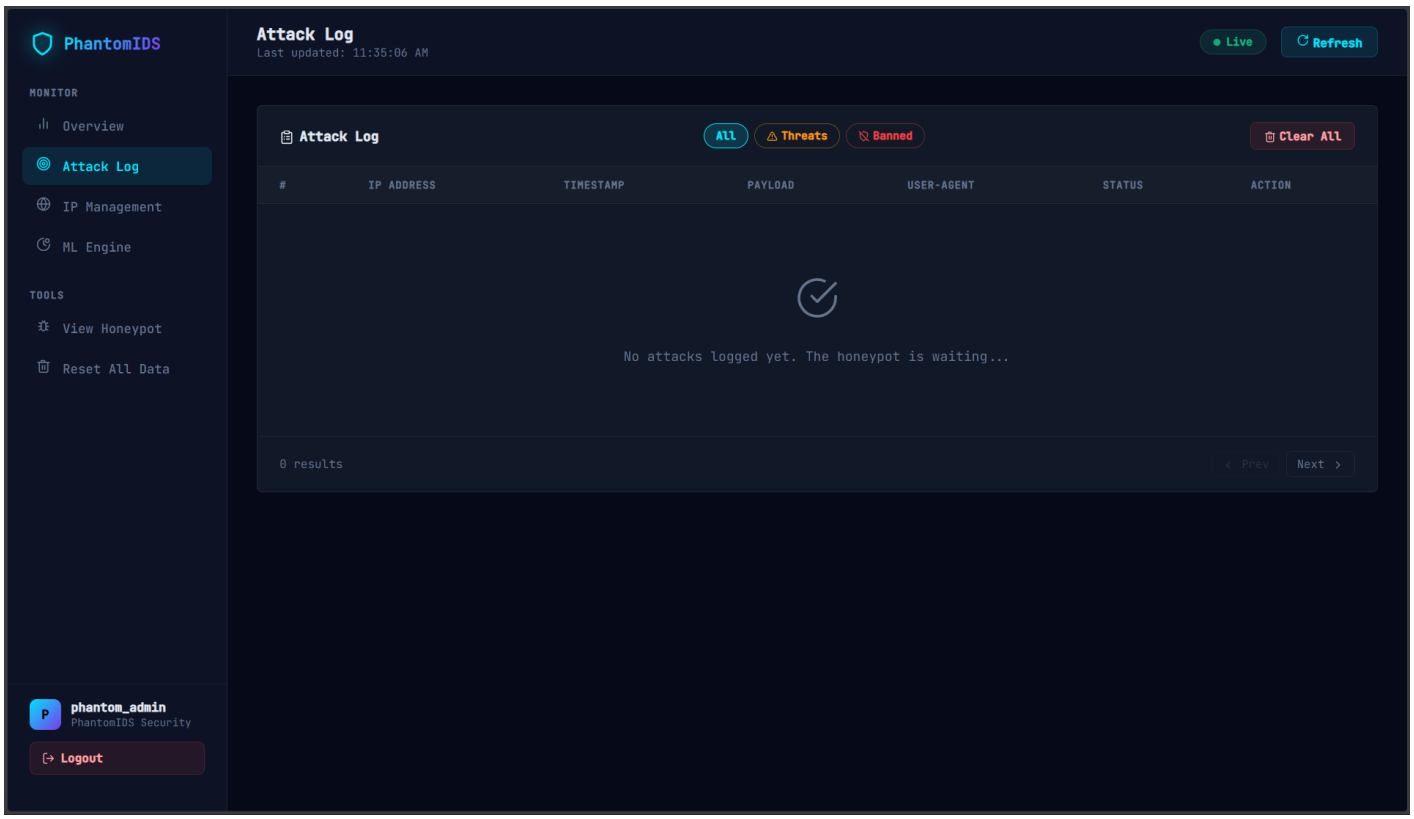

FIGURE 5.2.11 ADMIN DASHBOARD ATTCAK LOG SECTION (INITIAL STATE)

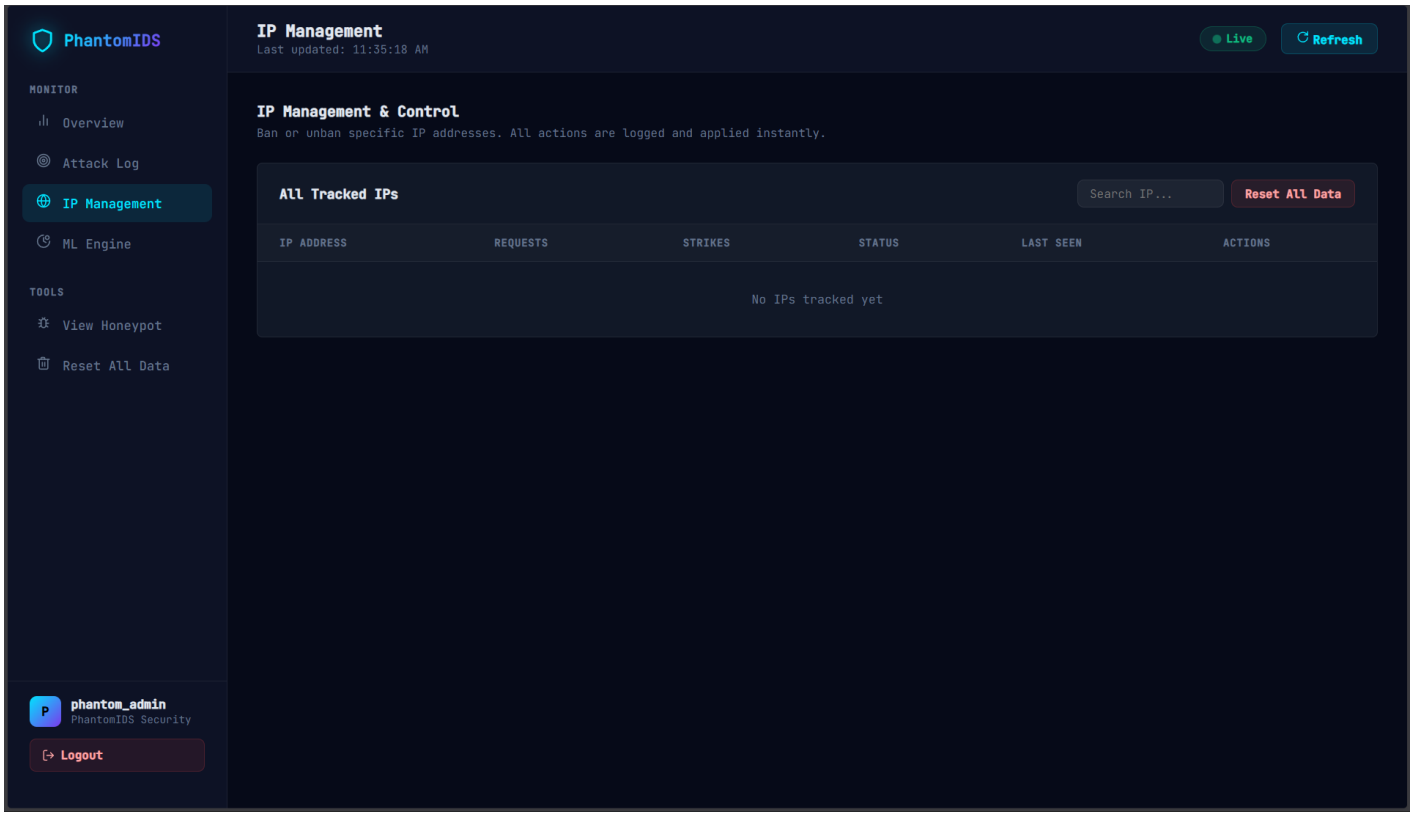

FIGURE 5.2.12 ADMIN DASHBOARD IP MANAGEMENT SECTION (INITIAL STATE)

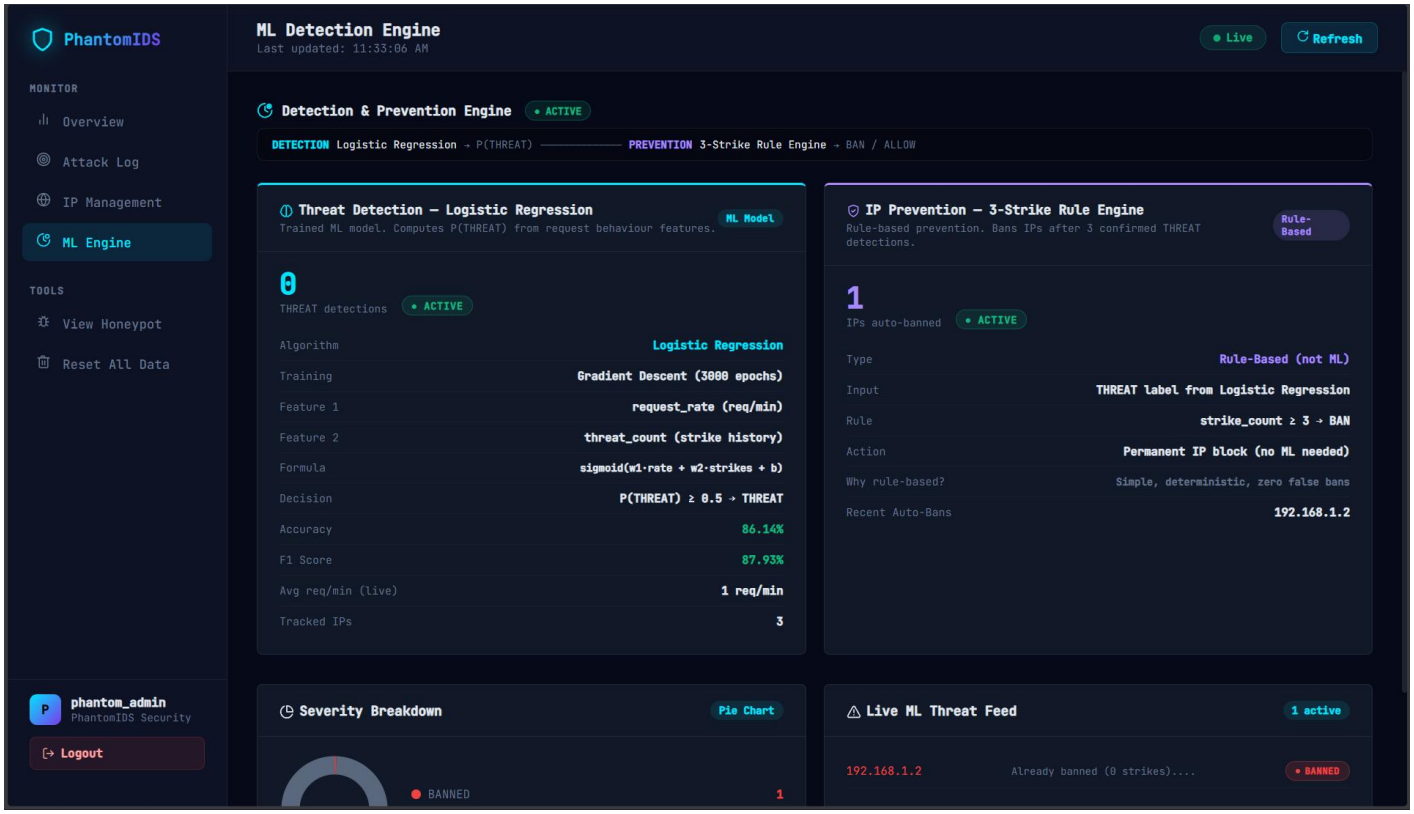

FIGURE 5.2.13 ADMIN DASHBOARD ML ENGINE SECTION (INITIAL STATE)

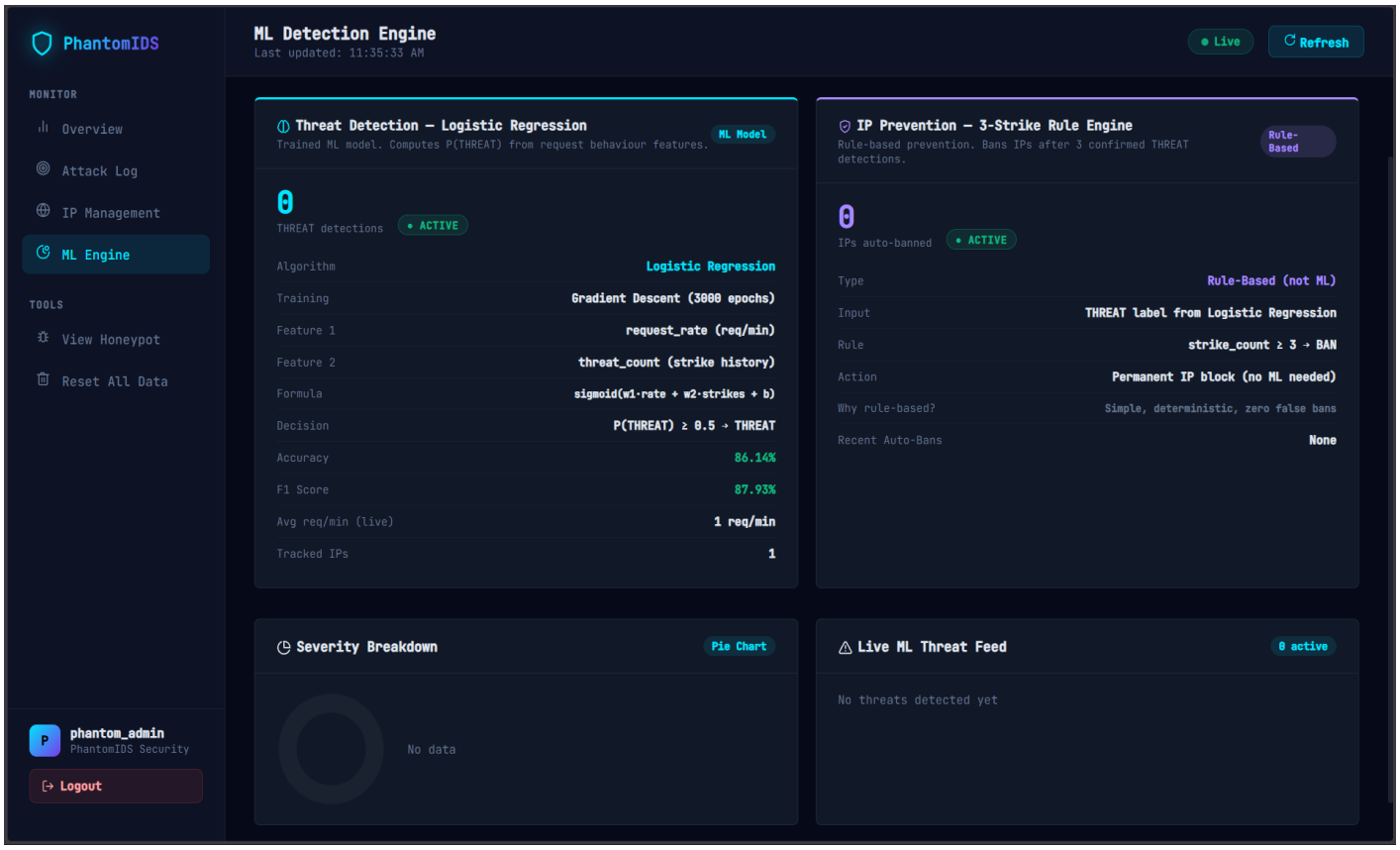

FIGURE 5.2.14 ADMIN DASHBOARD ML ENGINE SECTION STATS (INITIAL STATE)

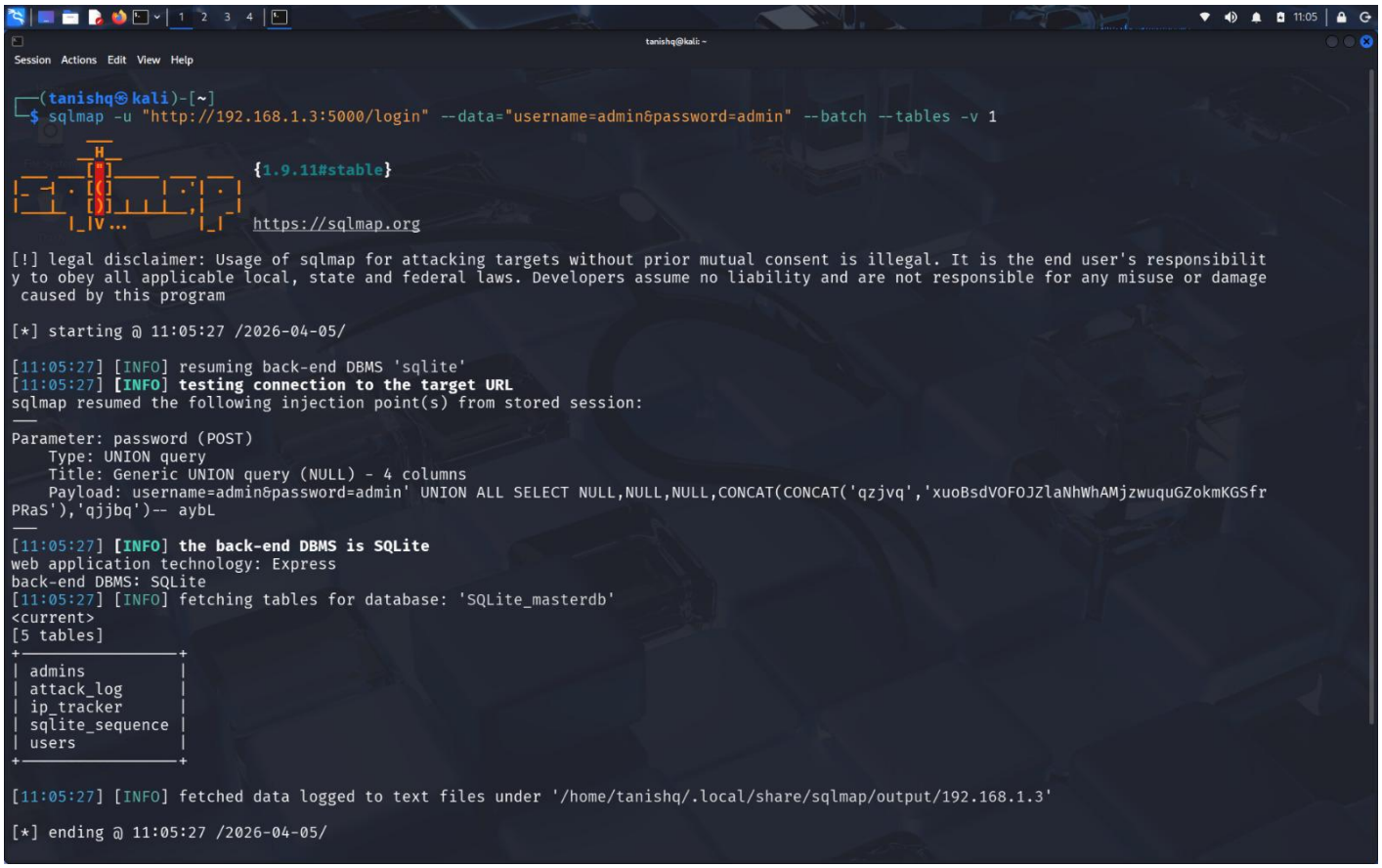

FIGURE 5.2.15 KALI LINUX TERMINAL RUNNING SQLMAP INJECTION DETECTION

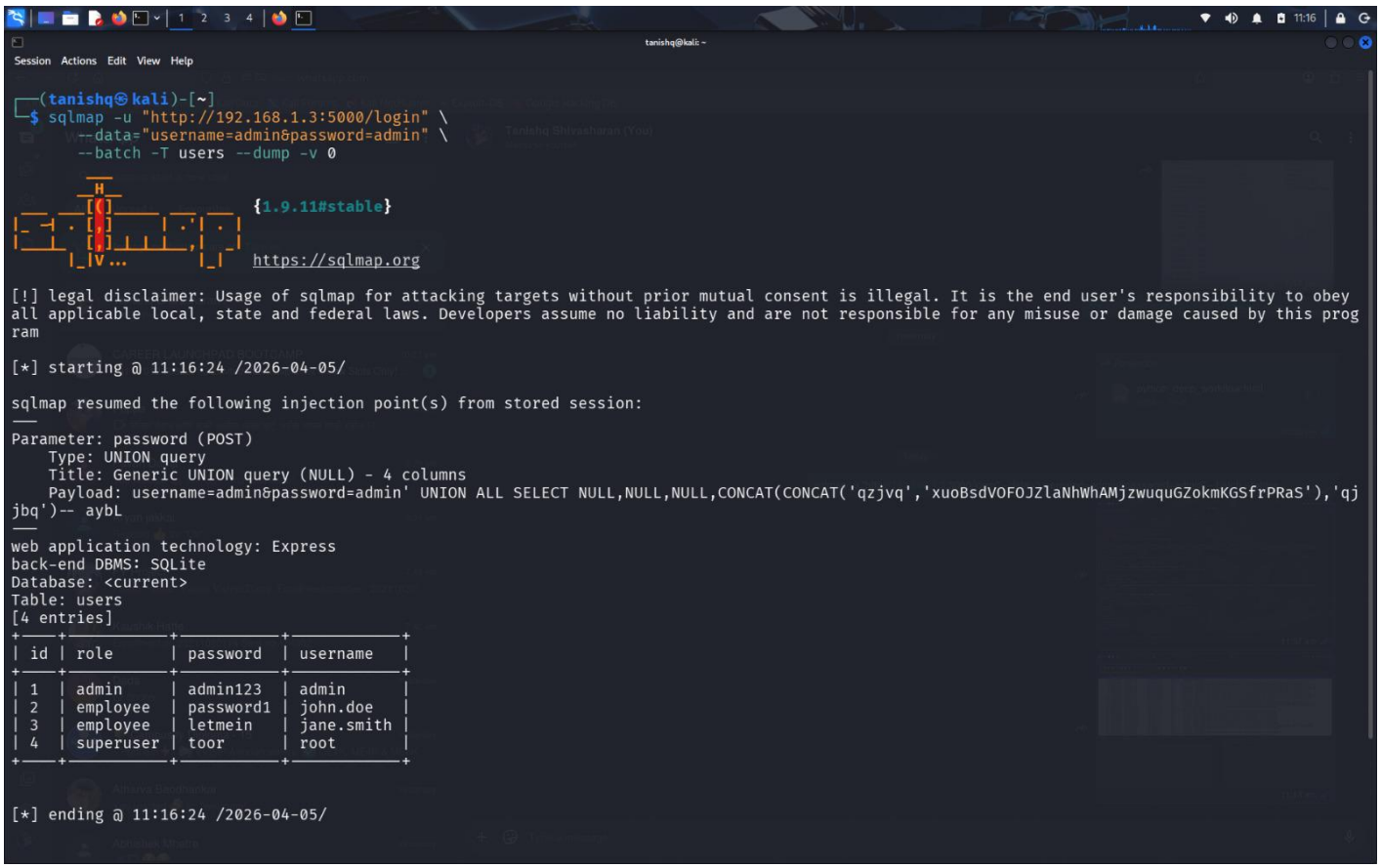

FIGURE 5.2.16 KALI LINUX TERMINAL SQLMAP DUMPING USERS TABLE

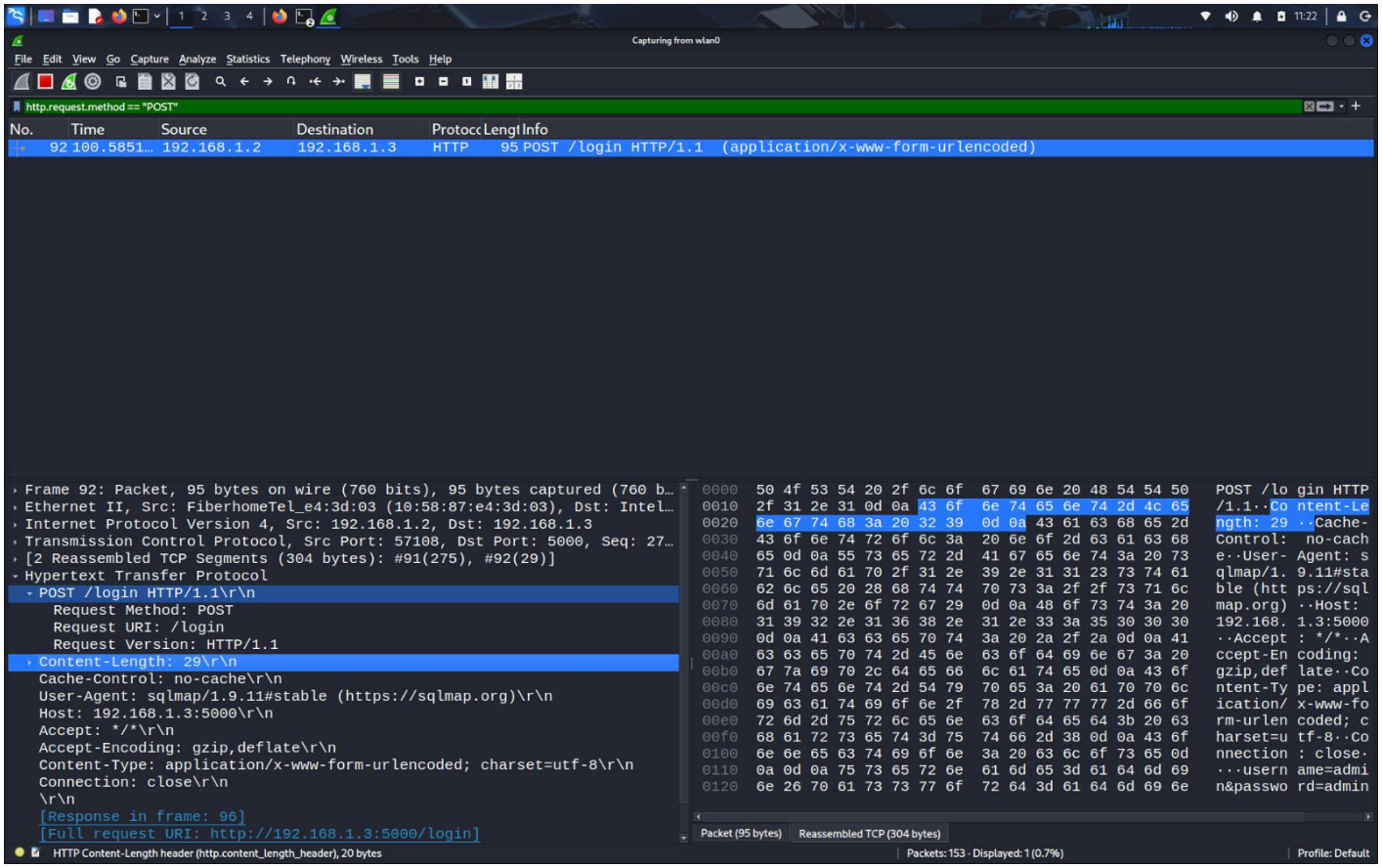

FIGURE 5.2.17 WIRESHARK CAPTURING SQL INJECTION PAYLOAD IN NETWORK PACKET

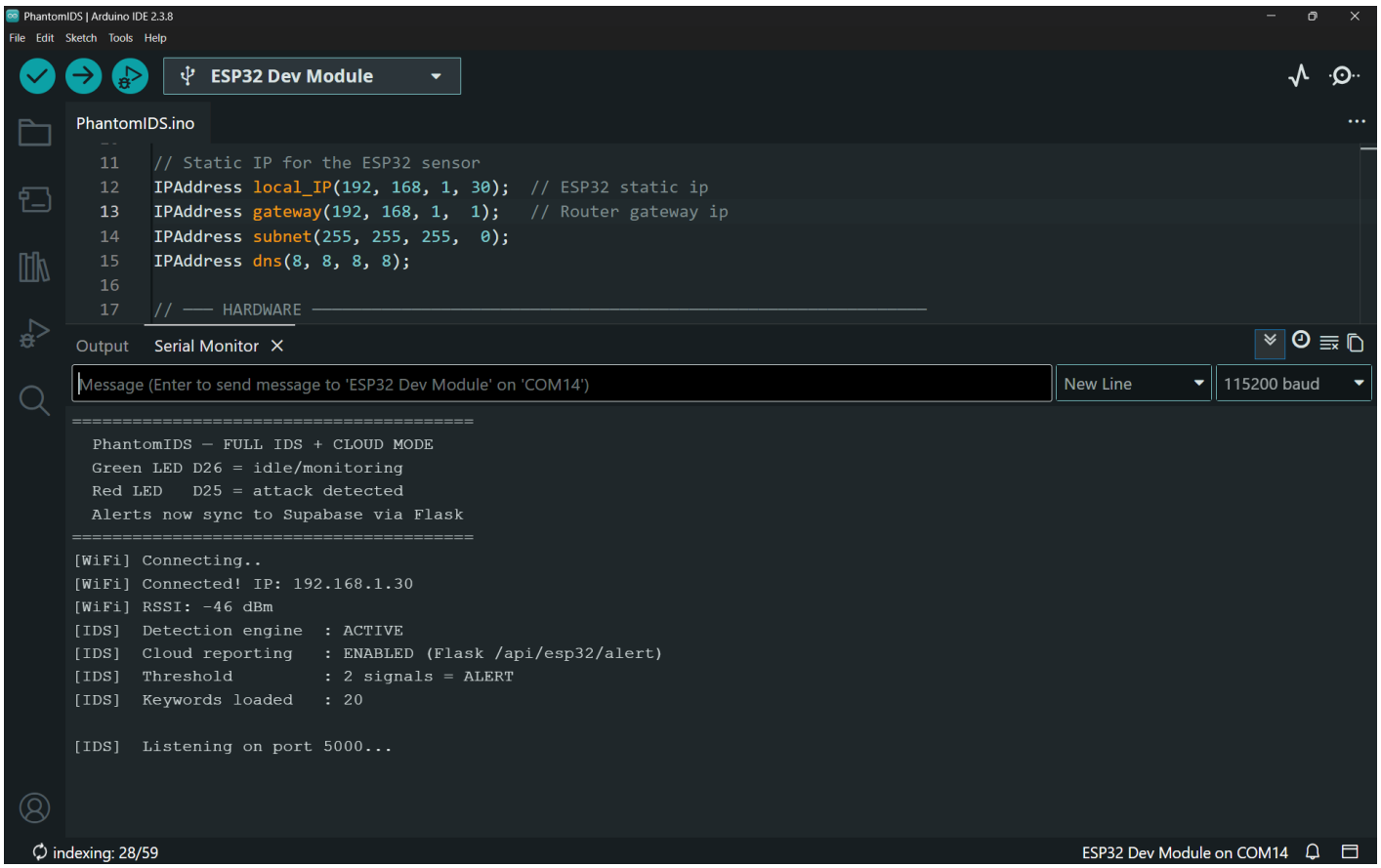

FIGURE 5.2.18 ESP32 SERIAL MONITOR STARTUP AND SYSTEM ARMED

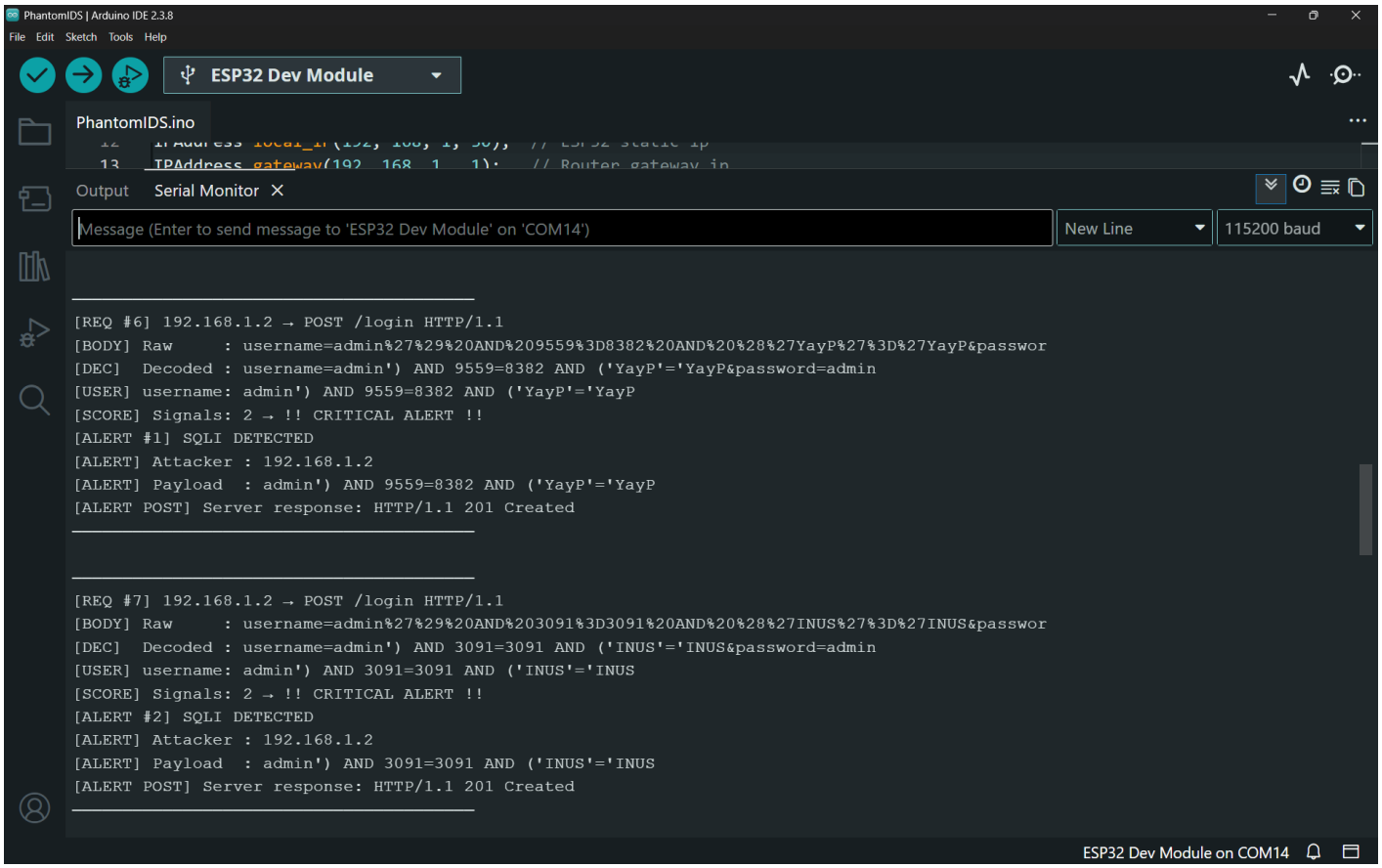

FIGURE 5.2.19 ESP32 SERIAL MONITOR DETECTING LIVE SQL INJECTION ATTACK

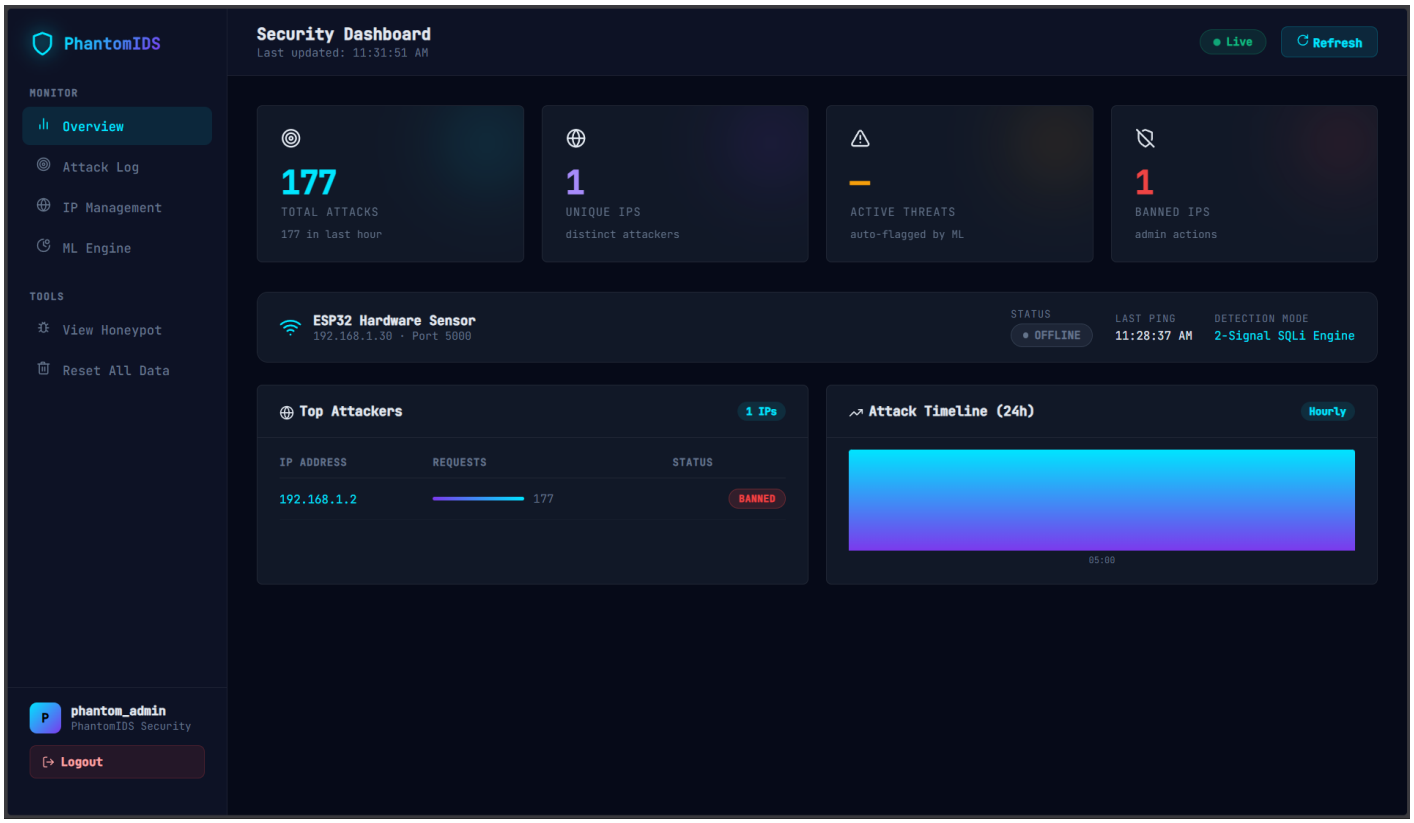

FIGURE 5.2.20 PHANTOMIDS LIVE DASHBOARD DISPLAYING REAL TIME ATTACK STATS

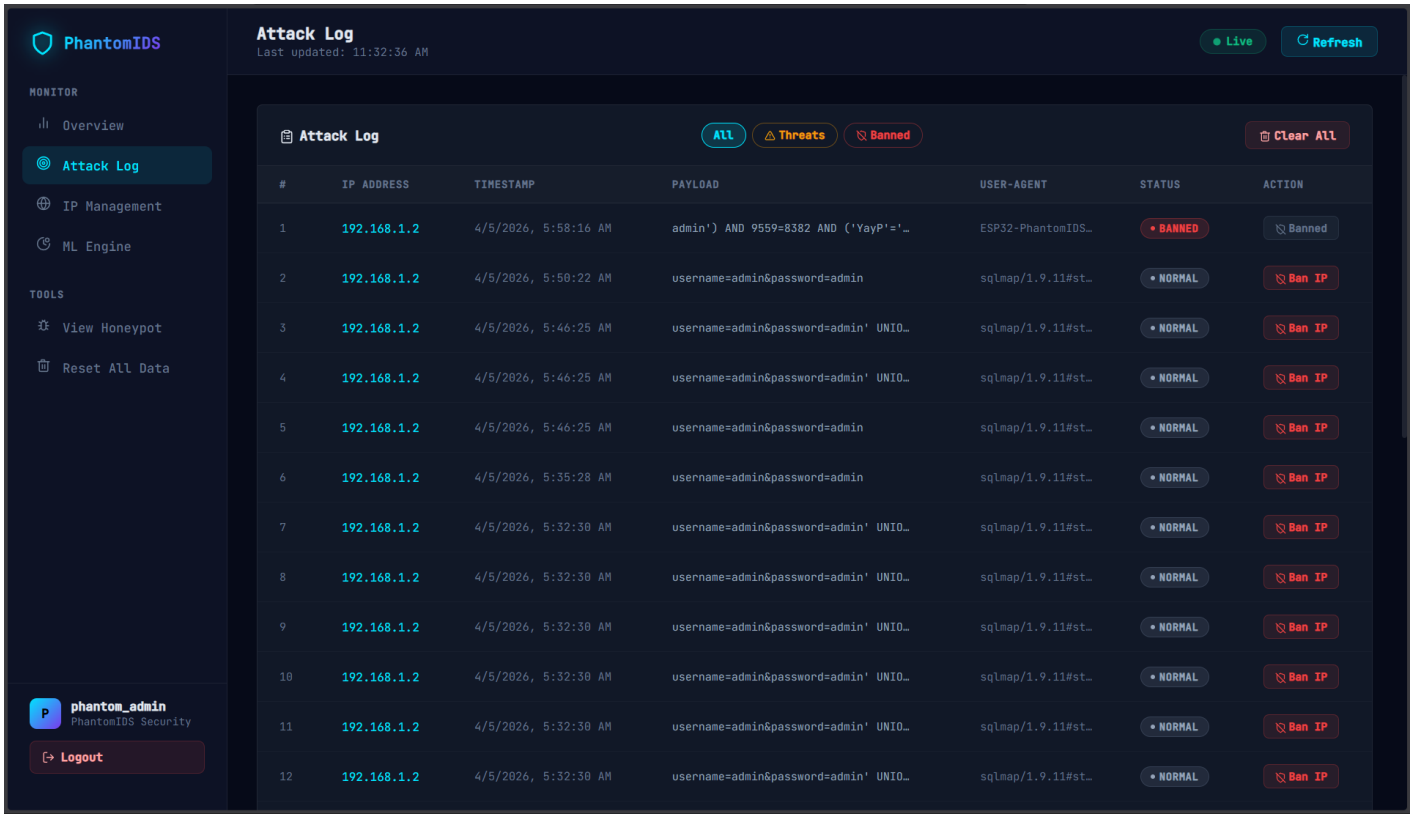

FIGURE 5.2.21 HONEYPOT ATTACK LOG SHOWING TIMESTAMPED INJECTION ATTEMPTS

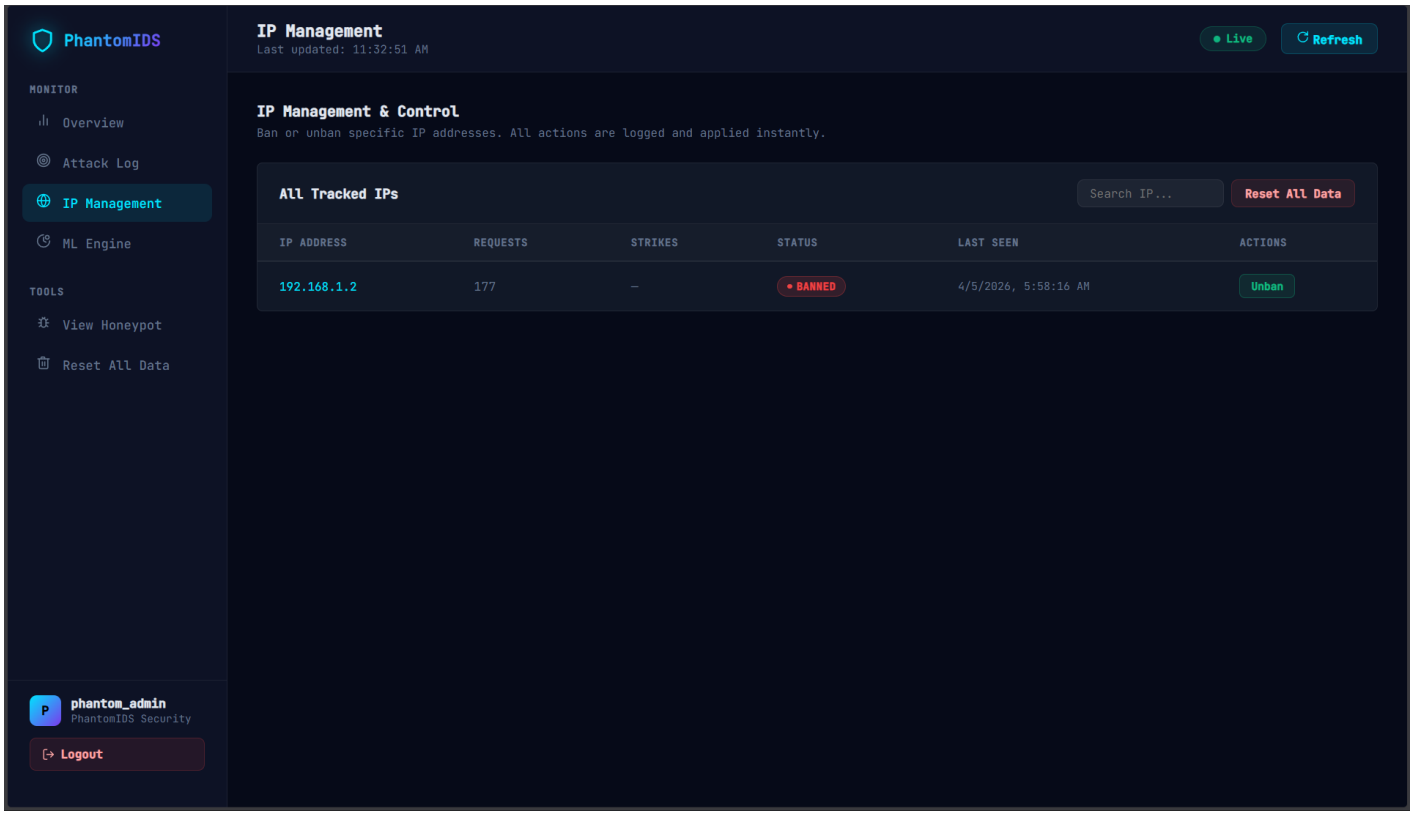

FIGURE 5.2.22 HONEYPOT ATTACK LOG SHOWING TIMESTAMPED INJECTION ATTEMPTS WITH IP MANAGEMENT

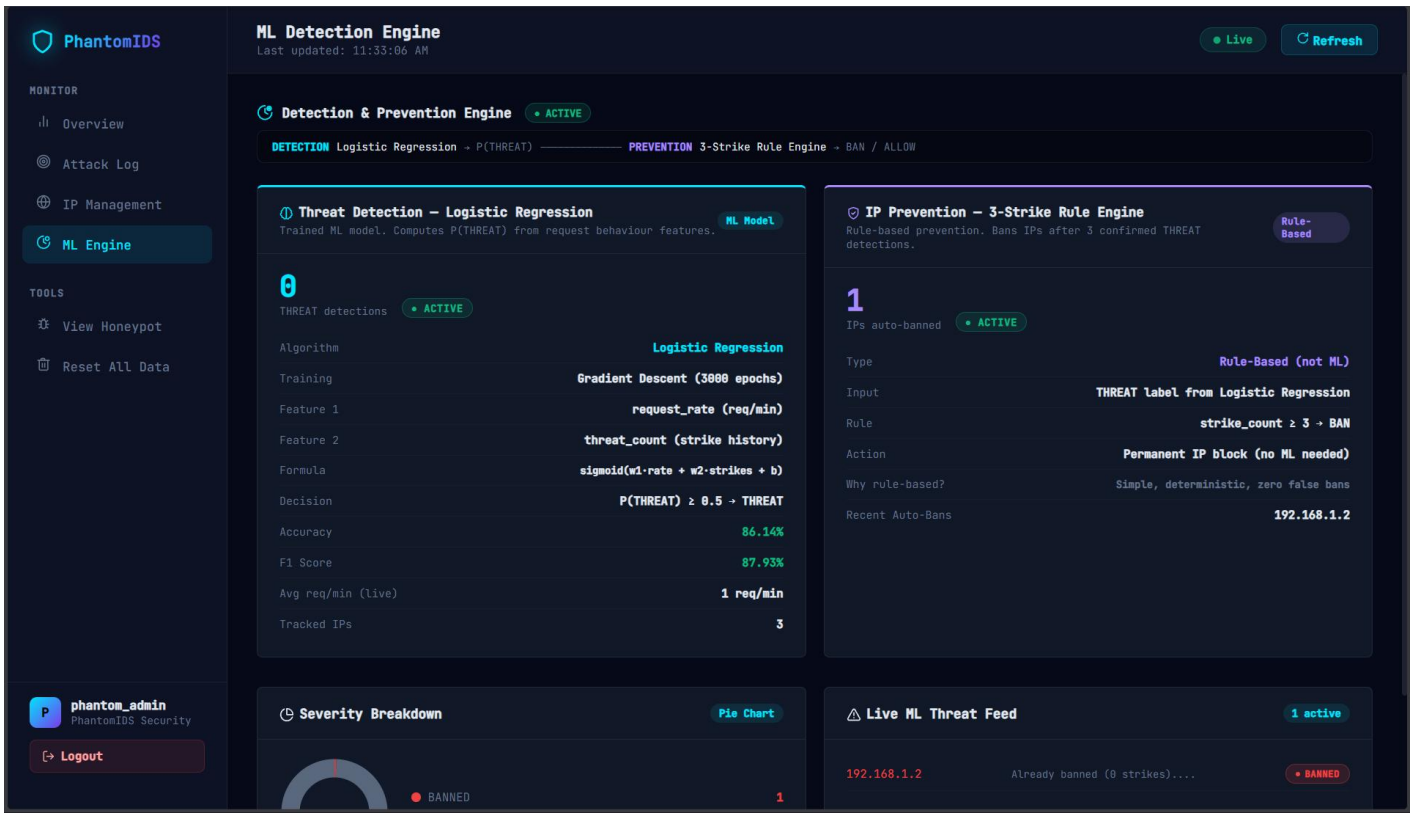

FIGURE 5.2.23 ML DETECTION ENGINE WITH THRESHOLD IP BANNING SYSTEM

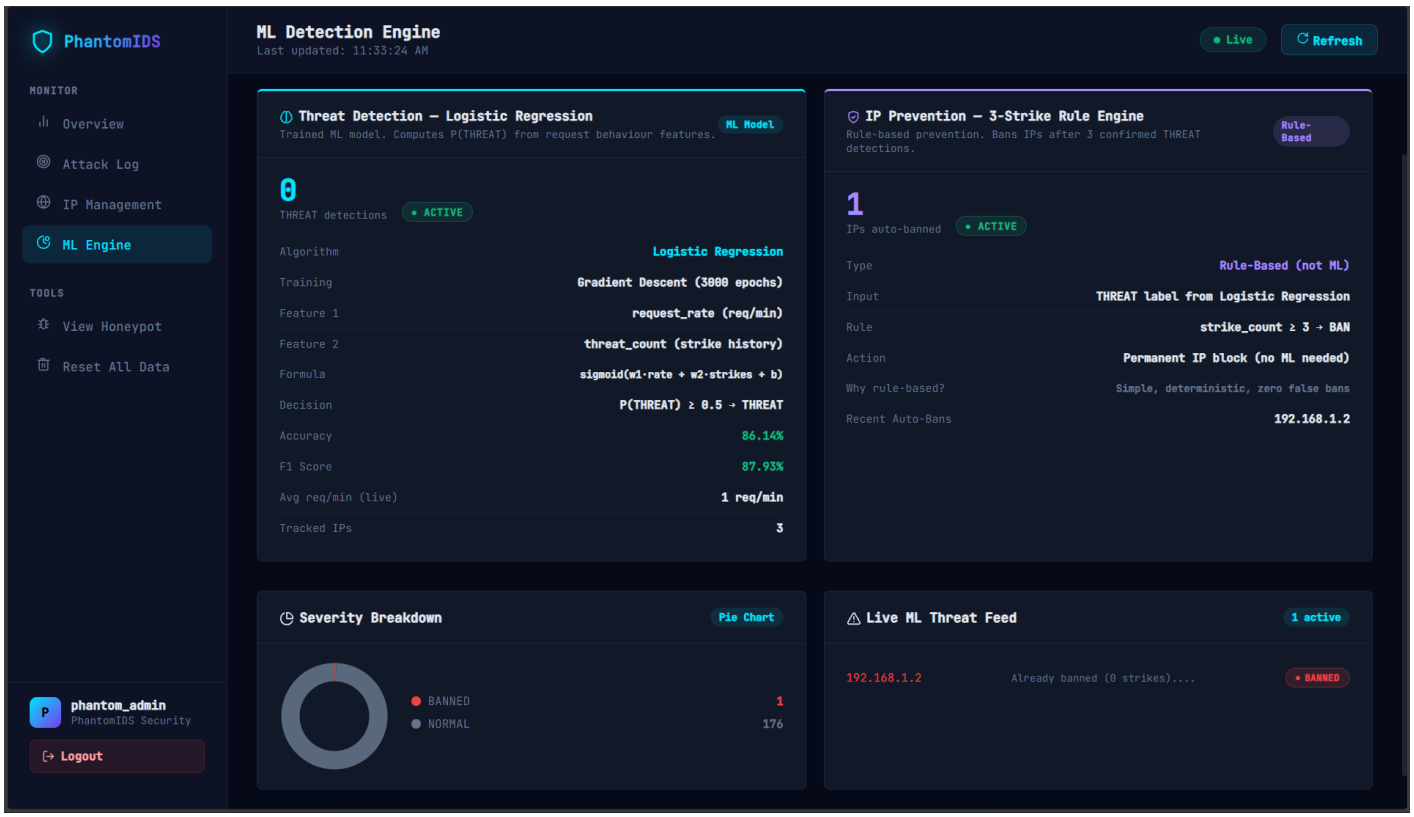

FIGURE 5.2.24 ML DETECTION ENGINE WITH THRESHOLD IP BANNING SYSTEM

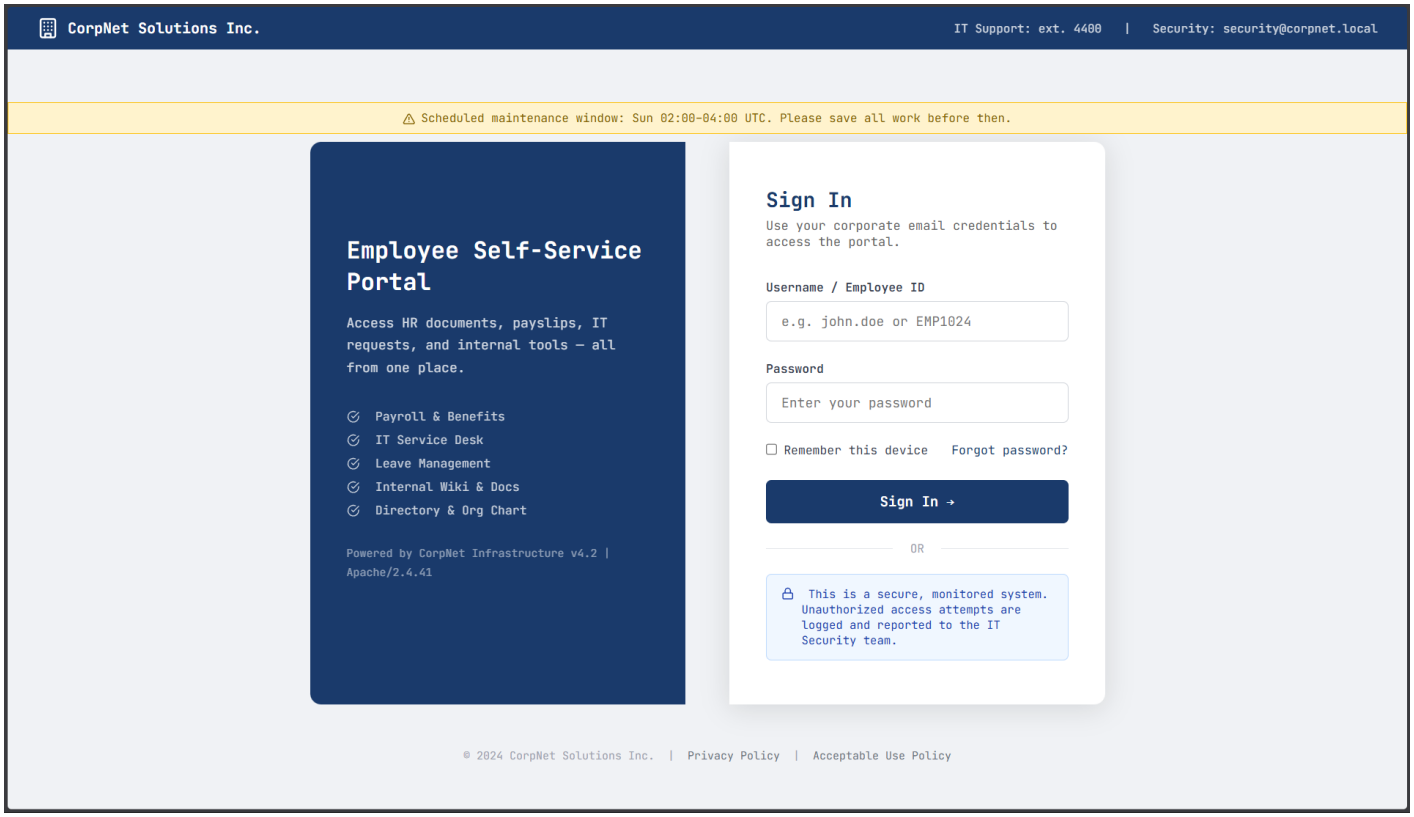

FIGURE 5.2.25 A VUNERABLE HONEYPOT LOGIN PAGE INITENTIONALLY MADE FOR TRAPPING AND LOGING ATTACKER'S

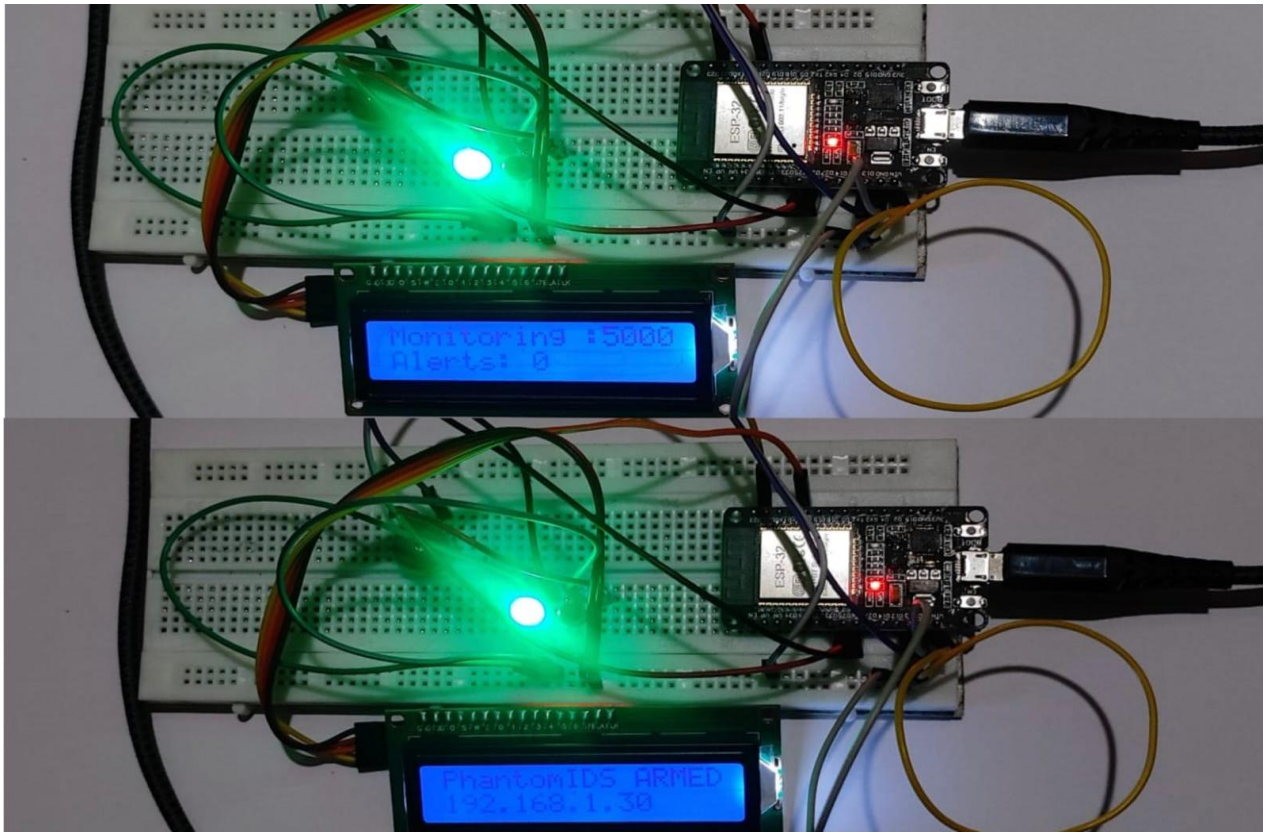

FIGURE 5.2.26 PHANTOMIDS HARDWARE IN MONITORIING STATE

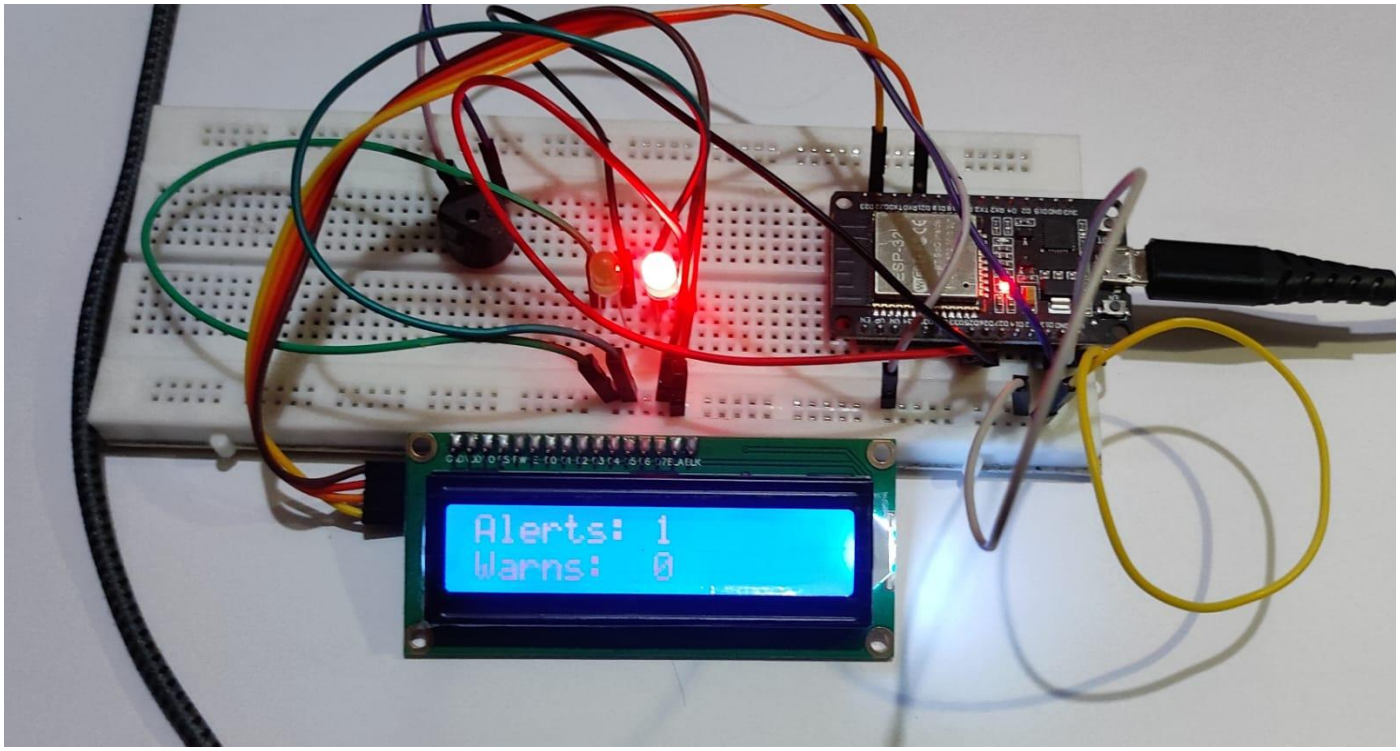

FIGURE 5.2.26 PHANTOMIDS HARDWARE ACTIVE BUZZER BEEPS 3 TIMES WITH RED LIGHT LIGHTING UP INDICATING AN SQLI ATTACK

5.3 CODE DETAILS

firmware/IDS_Detection.ino

```
#include <WiFi.h>
#include <LiquidCrystal_I2C.h>

//———CONFIGURATION———————————————————————————————————————
const char* SSID         = "<Wi-Fi Name>";
const char* PASSWORD     = "<Wi-Fi Password>";
const char* HONEYPOT_IP  = "192.168.1.3";
const int   HONEYPOT_PORT = 5000;
const int   LISTEN_PORT   = 5000;

// Static IP for the ESP32 sensor
IPAddress local_IP(192, 168, 1, 30); // ESP32 static ip
IPAddress gateway(192, 168, 1,  1);
IPAddress subnet(255, 255, 255,  0);
IPAddress dns(8, 8, 8, 8);

//———HARDWARE————————————————————————————————————————————
LiquidCrystal_I2C lcd(0x27, 16, 2);

#define BUZZER_PIN  13
#define LED_GREEN   26
#define LED_RED     25

WiFiServer server(LISTEN_PORT);

//———COUNTERS————————————————————————————————————————————
int totalRequests = 0;
int alertCount    = 0;
int warnCount     = 0;

//———DETECTION SIGNATURES (20 keywords)———————————————————
const char* SQL_KEYWORDS[] = {
  " OR ", " AND ", "UNION", "SELECT", "INSERT",
  "DROP", "DELETE", "UPDATE", "FROM", "WHERE",
  "SLEEP", "BENCHMARK", "HAVING", "ORDER BY",
  "--", "/*", "#", "1=1", "1 = 1", "xp_"
};
const int KEYWORD_COUNT = 20;

//———LED/BUZZER HELPERS——————————————————————————————————
void setIdle() {
```

```
  digitalWrite(LED_GREEN, HIGH);
  digitalWrite(LED_RED,   LOW);
}

void setWarning() {
  digitalWrite(LED_GREEN, LOW);
  delay(200);
  digitalWrite(LED_GREEN, HIGH);
}

void setAlert() {
  digitalWrite(LED_GREEN, LOW);
  digitalWrite(LED_RED,   HIGH);
}

void beep(int times, int ms = 80) {
  for (int i = 0; i < times; i++) {
    digitalWrite(BUZZER_PIN, HIGH); delay(ms);
    digitalWrite(BUZZER_PIN, LOW);  delay(ms);
  }
}

//——PAYLOAD ANALYSIS ————————————————————————————————
String urlDecode(const String& encoded) {
  String decoded = "";
  int len = encoded.length();
  for (int i = 0; i < len; i++) {
    if (encoded[i] == '%' && i + 2 < len) {
      char hex[3] = { encoded[i+1], encoded[i+2], '\0' };
      decoded += (char) strtol(hex, nullptr, 16);
      i += 2;
    } else if (encoded[i] == '+') {
      decoded += ' ';
    } else {
      decoded += encoded[i];
    }
  }
  return decoded;
}

bool hasQuote(const String& body) {
  return (body.indexOf("'") != -1 || body.indexOf("\"") != -1);
}

bool hasSQLKeyword(const String& body) {
  String upper = body;
  upper.toUpperCase();
  for (int i = 0; i < KEYWORD_COUNT; i++) {
    String kw = String(SQL_KEYWORDS[i]);
    kw.toUpperCase();
```

```
    if (upper.indexOf(kw) != -1) return true;
  }
  return false;
}

String extractBody(const String& raw) {
  int idx = raw.indexOf("\r\n\r\n");
  if (idx == -1) return "";
  return raw.substring(idx + 4);
}

String extractField(const String& body, const String& field) {
  String key = field + "=";
  int start = body.indexOf(key);
  if (start == -1) return "";
  start += key.length();
  int end = body.indexOf("&", start);
  if (end == -1) end = body.length();
  return body.substring(start, end);
}

int detectSQLi(const String& body, String& decodedOut) {
  if (body.length() == 0) return 0;
  decodedOut   = urlDecode(body);
  bool signal1 = hasQuote(decodedOut);
  bool signal2 = hasSQLKeyword(decodedOut);
  return (signal1 ? 1 : 0) + (signal2 ? 1 : 0);
}

//——POST ALERT TO NODE.JS SERVER ————————————————————————————
/**
 * After detecting a confirmed SQLi (score >= 2), the ESP32 POSTs a JSON
 * alert to the Node.js honeypot's /api/esp32/alert endpoint.
 */
void postAlertToServer(const String& attackerIP, const String& payload, int score) {
  WiFiClient client;
  if (!client.connect(HONEYPOT_IP, HONEYPOT_PORT)) {
    Serial.println("[ALERT POST] Could not reach server — skipping cloud report");
    return;
  }

  String safePayload = payload;
  safePayload.replace("\\", "\\\\");
  safePayload.replace("\"", "\\\"");

  String body =
    "{\"attacker_ip\":\"" + attackerIP + "\","
    "\"payload\":\""      + safePayload + "\","
    "\"score\":"          + String(score) + "}";
```

```
  String httpReq =
    "POST /api/esp32/alert HTTP/1.1\r\n"
    "Host: " + String(HONEYPOT_IP) + "\r\n"
    "Content-Type: application/json\r\n"
    "Content-Length: " + String(body.length()) + "\r\n"
    "Connection: close\r\n\r\n" + body;

  client.print(httpReq);

  unsigned long t = millis();
  while (client.connected() && (millis() - t < 2000)) {
    if (client.available()) {
      String line = client.readStringUntil('\n');
      if (line.startsWith("HTTP/1.1")) {
        Serial.println("[ALERT POST] Server response: " + line);
        break;
      }
    }
  }
  client.stop();
}

// ——— HEARTBEAT PING TO SERVER ————————————————————————————
/**
 * Sends a lightweight POST /api/esp32/ping every ~30 seconds so the
 * dashboard can show the hardware sensor as ONLINE.
 */
unsigned long lastPingTime = 0;
const unsigned long PING_INTERVAL = 30000; // 30 seconds

void sendHeartbeat() {
  if (millis() - lastPingTime < PING_INTERVAL) return;
  lastPingTime = millis();

  WiFiClient client;
  if (!client.connect(HONEYPOT_IP, HONEYPOT_PORT)) return;

  String httpReq =
    "POST /api/esp32/ping HTTP/1.1\r\n"
    "Host: " + String(HONEYPOT_IP) + "\r\n"
    "Content-Length: 0\r\n"
    "Connection: close\r\n\r\n";

  client.print(httpReq);
  delay(200);
  client.stop();
  Serial.println("[PING] Heartbeat sent to server ✓");
}

// ——— LCD + PHYSICAL ALERT ————————————————————————————————
```

```
void showAlert(int score, const String& attackerIP, const String& payload) {
  if (score >= 2) {
    setAlert();

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("!!SQLI DETECTED!");

    String ipLine = "IP:" + attackerIP;
    if (ipLine.length() > 16) ipLine = ipLine.substring(ipLine.length() - 16);
    lcd.setCursor(0, 1);
    lcd.print(ipLine);

    for (int i = 0; i < 3; i++) {
      digitalWrite(BUZZER_PIN, HIGH);
      digitalWrite(LED_RED,    HIGH);
      delay(120);
      digitalWrite(BUZZER_PIN, LOW);
      digitalWrite(LED_RED,    LOW);
      delay(120);
    }

    delay(2000);

    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Alerts: "); lcd.print(alertCount);
    lcd.setCursor(0, 1); lcd.print("Warns:  "); lcd.print(warnCount);
    delay(1500);

    setAlert();

  } else if (score == 1) {
    setWarning();

    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("WARN: Suspicious");
    String ipSnip = "IP:" + attackerIP.substring(attackerIP.lastIndexOf('.'));
    lcd.setCursor(0, 1); lcd.print(ipSnip);

    beep(1);
    delay(1000);
    setIdle();
  }
}

//                                    ——                                    SETUP
// ─────────────────────────────────────────────────────────────────────────

void setup() {
  Serial.begin(115200);
  delay(500);
```

```
pinMode(BUZZER_PIN, OUTPUT); digitalWrite(BUZZER_PIN, LOW);
pinMode(LED_GREEN,  OUTPUT); digitalWrite(LED_GREEN,  LOW);
pinMode(LED_RED,    OUTPUT); digitalWrite(LED_RED,    LOW);

// LED self-test
digitalWrite(LED_GREEN, HIGH); delay(400); digitalWrite(LED_GREEN, LOW);
digitalWrite(LED_RED,   HIGH); delay(400); digitalWrite(LED_RED,   LOW);
beep(1);

lcd.init();
lcd.backlight();
lcd.setCursor(0, 0); lcd.print("PhantomIDS v1.0 ");
lcd.setCursor(0, 1); lcd.print("Starting...     ");

WiFi.config(local_IP, gateway, subnet, dns);
WiFi.begin(SSID, PASSWORD);

Serial.println("=========================================");
Serial.println("  PhantomIDS — FULL IDS + CLOUD MODE");
Serial.println("  Green LED D26 = idle/monitoring");
Serial.println("  Red LED   D25 = attack detected");
Serial.println("  Alerts now sync to Supabase via Flask");
Serial.println("=========================================");
Serial.print("[WiFi] Connecting");

int att = 0;
while (WiFi.status() != WL_CONNECTED) {
  delay(500); Serial.print(".");
  if (++att > 30) {
    Serial.println("\n[FATAL] WiFi failed — check credentials");
    lcd.setCursor(0, 1); lcd.print("WiFi FAILED!    ");
    while (true) delay(1000);
  }
}

Serial.println("\n[WiFi] Connected! IP: " + WiFi.localIP().toString());
Serial.print("[WiFi] RSSI: "); Serial.print(WiFi.RSSI()); Serial.println(" dBm");
Serial.println("[IDS]  Detection engine  : ACTIVE");
Serial.println("[IDS]  Cloud reporting   : ENABLED (Flask /api/esp32/alert)");
Serial.println("[IDS]  Threshold         : 2 signals = ALERT");
Serial.println("[IDS]  Keywords loaded   : " + String(KEYWORD_COUNT));
Serial.println("\n[IDS]  Listening on port " + String(LISTEN_PORT) + "...\n");

server.begin();
setIdle();

// Send immediate boot ping so dashboard shows ONLINE right away
sendHeartbeat();
lastPingTime = 0; // reset so next ping fires after PING_INTERVAL
```

```
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("PhantomIDS ARMED");
  lcd.setCursor(0, 1); lcd.print("192.168.1.30    ");
  beep(2);
  delay(1500);

  lcd.setCursor(0, 0); lcd.print("Monitoring :5000");
  lcd.setCursor(0, 1); lcd.print("Alerts: 0       ");
}

// ——— MAIN LOOP ————————————————————————————————————————————

void loop() {
  WiFiClient client = server.available();
  if (!client) return;

  totalRequests++;
  String attackerIP = client.remoteIP().toString();

  // Read full HTTP request (up to 4KB, 3s timeout)
  String rawRequest = "";
  unsigned long t0 = millis();
  while (client.connected() && (millis() - t0 < 3000)) {
    while (client.available()) {
      rawRequest += (char)client.read();
      if (rawRequest.length() > 4096) break;
    }
    if (rawRequest.indexOf("\r\n\r\n") != -1) {
      delay(10);
      while (client.available()) rawRequest += (char)client.read();
      break;
    }
  }

  String firstLine = rawRequest.substring(0, rawRequest.indexOf("\r\n"));
  String body      = extractBody(rawRequest);
  bool   isPOST    = firstLine.startsWith("POST");

  Serial.println("————————————————————————————————————————————————————————————");
  Serial.print("[REQ #"); Serial.print(totalRequests);
  Serial.print("] "); Serial.print(attackerIP);
  Serial.print(" → "); Serial.println(firstLine);

  String decoded = "";
  int    score   = 0;

  if (isPOST && body.length() > 0) {
    score = detectSQLi(body, decoded);
    String userPayload = extractField(decoded, "username");
```

```
    Serial.print("[BODY] Raw     : "); Serial.println(body.substring(0, 80));
    Serial.print("[DEC]  Decoded : "); Serial.println(decoded.substring(0, 80));
    Serial.print("[USER] username: "); Serial.println(userPayload.substring(0, 60));
    Serial.print("[SCORE] Signals: "); Serial.print(score);

    if (score >= 2) {
      alertCount++;
      Serial.println(" → !! CRITICAL ALERT !!");
      Serial.print("[ALERT #"); Serial.print(alertCount); Serial.println("] SQLI DETECTED");
      Serial.print("[ALERT] Attacker : "); Serial.println(attackerIP);
      Serial.print("[ALERT] Payload  : "); Serial.println(userPayload.substring(0, 60));

      showAlert(2, attackerIP, userPayload);

      // ── NEW: Report to Node.js server ──────────────────────────
      postAlertToServer(attackerIP, userPayload, score);

    } else if (score == 1) {
      warnCount++;
      Serial.println(" → WARNING (1 signal)");
      showAlert(1, attackerIP, userPayload);

    } else {
      Serial.println(" → CLEAN");
      setIdle();
    }
  } else {
    Serial.println("[SKIP] GET — no body");
    setIdle();
  }

  // Forward raw request to Flask honeypot
  WiFiClient hp;
  String hpResponse = "";

  if (hp.connect(HONEYPOT_IP, HONEYPOT_PORT)) {
    // Inject real attacker IP so Node.js logs the correct source
    String modifiedRequest = rawRequest;
    int headerEnd = modifiedRequest.indexOf("\r\n\r\n");
    if (headerEnd != -1) {
      String xForwarded = "\r\nX-Forwarded-For: " + attackerIP;
                modifiedRequest = modifiedRequest.substring(0, headerEnd) + xForwarded +
modifiedRequest.substring(headerEnd);
    }
    hp.print(modifiedRequest);
    delay(400);
    unsigned long t1 = millis();
    while (hp.connected() && (millis() - t1 < 3000)) {
      while (hp.available()) {
```

```
      hpResponse += (char)hp.read();
      if (hpResponse.length() > 4096) break;
    }
   }
   hp.stop();
  } else {
   hpResponse = "HTTP/1.1 502 Bad Gateway\r\nContent-Length: 0\r\n\r\n";
   Serial.println("[FWD]  WARNING: honeypot unreachable!");
  }

  client.print(hpResponse);
  client.stop();

  // Update LCD status line
  if (score < 2) {
   lcd.setCursor(0, 0); lcd.print("Monitoring :5000");
   String line2 = "A:" + String(alertCount) +
                  " W:" + String(warnCount)  +
                  " R:" + String(totalRequests);
   while (line2.length() < 16) line2 += " ";
   lcd.setCursor(0, 1); lcd.print(line2.substring(0, 16));
  }

  Serial.println("──────────────────────────────────────────────────────────────────\n");

  // Send heartbeat ping to keep dashboard sensor status ONLINE
  sendHeartbeat();
}
```

server/src/middleware/threatDetector.js

```
'use strict';

const db = require('../db/database');
const { ML_MODEL_1, ML_MODEL_2 } = require('../ml/models');

const WINDOW_MS   = ML_MODEL_1.hyperparams.WINDOW_MS;
const BAN_STRIKES = ML_MODEL_2.hyperparams.BAN_STRIKES;

function threatDetector(req, res, next) {
  const ip  = req.ip || req.connection.remoteAddress || 'unknown';
  const now = Date.now();

  try {
    let tracker = db.prepare('SELECT * FROM ip_tracker WHERE ip = ?').get(ip);

    if (!tracker) {
      db.prepare(`
        INSERT INTO ip_tracker (ip, request_count, window_start, status, threat_count)
```

```
      VALUES (?, 1, datetime('now'), 'NORMAL', 0)
    `).run(ip);
    req.ipStatus = 'NORMAL';
    return next();
  }

  if (tracker.status === 'BANNED') {
    req.ipStatus = 'BANNED';
    return next();
  }

  const elapsed = now - new Date(tracker.window_start).getTime();

  if (elapsed > WINDOW_MS) {
    db.prepare(`
      UPDATE ip_tracker SET request_count = 1, window_start = datetime('now') WHERE ip = ?
    `).run(ip);
    req.ipStatus = tracker.status;
    return next();
  }

  const newCount = tracker.request_count + 1;
  db.prepare('UPDATE ip_tracker SET request_count = ? WHERE ip = ?').run(newCount, ip);

  const model1 = ML_MODEL_1.predict({
    requestCount:  newCount,
    windowMs:      elapsed,
    threatCount:   tracker.threat_count || 0,
    currentStatus: tracker.status,
  });

  if (model1.label === 'THREAT' && tracker.status === 'NORMAL') {
    const newStrikes = (tracker.threat_count || 0) + 1;
    db.prepare('UPDATE ip_tracker SET threat_count = ? WHERE ip = ?').run(newStrikes, ip);

    const model2 = ML_MODEL_2.predict({
      threatStrikes:  newStrikes,
      model1Decision: model1.label,
      currentStatus:  tracker.status,
    });

    if (model2.action === 'BAN_IP') {
      db.prepare("UPDATE ip_tracker SET status = 'BANNED' WHERE ip = ?").run(ip);
      db.prepare("UPDATE attack_log SET status = 'BANNED' WHERE ip = ?").run(ip);
        console.log(`[ML-Model2] AUTO-BANNED: ${ip} | Strike ${newStrikes}/${BAN_STRIKES} |
Confidence: ${(model2.confidence * 100).toFixed(0)}%`);
      req.ipStatus = 'BANNED';
    } else {
      db.prepare("UPDATE ip_tracker SET status = 'THREAT' WHERE ip = ?").run(ip);
```

```
        db.prepare("UPDATE attack_log SET status = 'THREAT' WHERE ip = ? AND status = 'NORMAL'").run(ip);
        console.log(`[ML-Model1] THREAT: ${ip} | ${newCount} req/${(elapsed/1000).toFixed(0)}s | Strike ${newStrikes}/${BAN_STRIKES} | Confidence: ${(model1.confidence * 100).toFixed(0)}%`);
       req.ipStatus = 'THREAT';
      }
    } else {
      req.ipStatus = tracker.status;
    }

  } catch (err) {
    console.error('[ThreatDetector] Error:', err.message);
  }

  next();
}

module.exports = threatDetector;
```

server/src/models.js

```
'use strict';

const fs   = require('fs');
const path = require('path');

const MODEL_PATH = path.join(__dirname, 'model.json');

function buildFallbackModel() {
  return {
    algorithm:  'Logistic Regression (fallback weights)',
    version:    '2.0.0-fallback',
    weights:    [4.948, 3.853],
    bias:       -2.044,
    norm:       { requestRate: { min: 1, max: 80 }, threatCount: { min: 0, max: 3 } },
    metrics:    { accuracy: 86.14, precision: 87.93, recall: 87.93, f1: 87.93 },
    classificationThreshold: 0.5,
  };
}

function loadModel() {
  try {
    if (!fs.existsSync(MODEL_PATH)) {
      console.warn('[ML-Model1] model.json not found — using fallback weights. Run: npm run train');
      return buildFallbackModel();
    }

    const saved = JSON.parse(fs.readFileSync(MODEL_PATH, 'utf8'));
```

```
    if (!saved.weights || saved.weights.length < 2 || saved.bias === undefined || !saved.norm) {
      throw new Error('model.json is malformed — missing weights/bias/norm');
    }

    console.log(`[ML-Model1] Logistic Regression loaded`);
    console.log(`           w1=${saved.weights[0]}  w2=${saved.weights[1]}  bias=${saved.bias}`);
    console.log(`           Accuracy: ${saved.metrics?.accuracy}%  F1: ${saved.metrics?.f1}%`);
    console.log(`           Trained on ${saved.trainingSamples} samples (${saved.trainedAt})`);

    return saved;
  } catch (err) {
    console.error('[ML-Model1] Load error:', err.message, '— using fallback weights.');
    return buildFallbackModel();
  }
}

const SAVED_MODEL = loadModel();

/** sigmoid(z) = 1 / (1 + e^-z) — maps any real number to (0, 1) */
function sigmoid(z) {
  return 1 / (1 + Math.exp(-z));
}

/** Normalizes value to [0, 1] using the same bounds recorded during training */
function minMaxNormalize(value, min, max) {
  const range   = max - min || 1;
  const clamped = Math.max(min, Math.min(max, value));
  return (clamped - min) / range;
}

const ML_MODEL_1 = {
  name:        'Behavioural Anomaly Detector',
  algorithm:   'Logistic Regression (Binary Cross-Entropy)',
  description: 'Computes P(THREAT) via sigmoid over normalized request_rate and threat_count.',
  version:     SAVED_MODEL.version || '2.0.0',

  hyperparams: {
    WEIGHTS:          SAVED_MODEL.weights,
    BIAS:             SAVED_MODEL.bias,
    THRESHOLD:        SAVED_MODEL.classificationThreshold || 0.5,
    NORM_REQUEST_RATE: SAVED_MODEL.norm.requestRate,
    NORM_THREAT_COUNT: SAVED_MODEL.norm.threatCount,
    TRAINING_ACCURACY: SAVED_MODEL.metrics?.accuracy,
    TRAINING_F1:      SAVED_MODEL.metrics?.f1,
    NORMAL_LABEL:     'NORMAL',
    THREAT_LABEL:     'THREAT',
    WINDOW_MS:        60 * 1000,
    THREAT_THRESHOLD:  20,
  },
```

```
  /**
   * Classifies an IP as NORMAL or THREAT using logistic regression.
   * z = w1·norm(request_rate) + w2·norm(threat_count) + bias
   * P(THREAT) = sigmoid(z)
   *
   * @param {object} features
   * @param {number} features.requestCount  - requests in current window
   * @param {number} features.windowMs      - elapsed window time in ms
   * @param {number} features.threatCount   - accumulated strike history
   * @param {string} features.currentStatus - current DB status
   */
  predict({ requestCount, windowMs, currentStatus, threatCount = 0 }) {
    const hp = ML_MODEL_1.hyperparams;

    if (currentStatus === 'BANNED') {
      return { label: 'BANNED', probability: 1.0, confidence: 1.0, features: {}, reason: 'IP is permanently banned.', modelUsed: hp.algorithm };
    }

    const elapsedMin  = Math.max(windowMs / 60000, 1 / 60);
    const requestRate = requestCount / elapsedMin;
        const x1_norm = minMaxNormalize(requestRate, hp.NORM_REQUEST_RATE.min, hp.NORM_REQUEST_RATE.max);
        const x2_norm = minMaxNormalize(threatCount, hp.NORM_THREAT_COUNT.min, hp.NORM_THREAT_COUNT.max);
    const z           = hp.WEIGHTS[0] * x1_norm + hp.WEIGHTS[1] * x2_norm + hp.BIAS;
    const probability = sigmoid(z);
    const isThreat    = probability >= hp.THRESHOLD;
    const label       = isThreat ? hp.THREAT_LABEL : hp.NORMAL_LABEL;
    const confidence  = parseFloat((Math.abs(probability - 0.5) * 2).toFixed(3));

    return {
      label,
      probability: parseFloat(probability.toFixed(4)),
      confidence,
      features: {
        requestCount,
        requestRate:     parseFloat(requestRate.toFixed(2)),
        requestRateNorm: parseFloat(x1_norm.toFixed(4)),
        threatCount,
        threatCountNorm: parseFloat(x2_norm.toFixed(4)),
        linearScore_z:   parseFloat(z.toFixed(4)),
        windowSeconds:   parseFloat((windowMs / 1000).toFixed(1)),
        threshold:       hp.THRESHOLD,
        currentStatus,
      },
      reason: isThreat
        ? `P(THREAT)=${(probability*100).toFixed(1)}% >= 50% | rate=${requestRate.toFixed(1)} req/min, strikes=${threatCount}`
        : `P(THREAT)=${(probability*100).toFixed(1)}% < 50% | rate=${requestRate.toFixed(1)} req/min`,
```

```
    modelUsed: ML_MODEL_1.algorithm,
   };
  },
};

const ML_MODEL_2 = {
  name:        '3-Strike Auto-Ban Classifier',
  algorithm:   'Accumulative Strike Committee (Ensemble)',
  description: 'Counts Model 1 THREAT votes per IP. After 3 strikes → permanent ban.',
  version:     '2.0.0',

  hyperparams: {
   BAN_STRIKES:  3,
   THREAT_LABEL: 'THREAT',
   BAN_LABEL:    'BANNED',
  },

  /**
   * @param {object} features
   * @param {number} features.threatStrikes  - accumulated Model 1 THREAT votes
   * @param {string} features.model1Decision - current Model 1 label
   * @param {string} features.currentStatus  - current DB status
   */
  predict({ threatStrikes, model1Decision, currentStatus }) {
   const { BAN_STRIKES, THREAT_LABEL, BAN_LABEL } = ML_MODEL_2.hyperparams;

   if (currentStatus === 'BANNED') {
    return { label: BAN_LABEL, confidence: 1.0, features: {}, reason: `Already banned (${threatStrikes}
strikes).`, action: 'NONE' };
   }

   if (model1Decision === THREAT_LABEL) {
    const newStrikes = threatStrikes + 1;
    const shouldBan  = newStrikes >= BAN_STRIKES;
    return {
     label:      shouldBan ? BAN_LABEL : THREAT_LABEL,
     confidence: shouldBan ? 1.0 : parseFloat((newStrikes / BAN_STRIKES).toFixed(3)),
          features:      { previousStrikes: threatStrikes, newStrikes, banThreshold: BAN_STRIKES,
votesRemaining: Math.max(0, BAN_STRIKES - newStrikes) },
      reason:     shouldBan
       ? `Strike ${newStrikes}/${BAN_STRIKES} — AUTO-BAN triggered.`
       : `Strike ${newStrikes}/${BAN_STRIKES} — ${BAN_STRIKES - newStrikes} more to ban.`,
     action: shouldBan ? 'BAN_IP' : 'FLAG_THREAT',
    };
   }

   return {
    label:      currentStatus || 'NORMAL',
    confidence: 1.0,
    features:   { strikes: threatStrikes, banThreshold: BAN_STRIKES },
```

```
      reason:     'Model 1 classified NORMAL — no action.',
      action:     'NONE',
    };
  },
};

/**
 * Runs both models in sequence on an ip_tracker record.
 * @returns {{ ip, model1, model2, finalDecision, timestamp }}
 */
function runMLPipeline(ipRecord, now = Date.now()) {
  const windowMs = Math.max(now - new Date(ipRecord.window_start).getTime(), 0);

  const model1Result = ML_MODEL_1.predict({
    requestCount:  ipRecord.request_count || 0,
    windowMs,
    threatCount:   ipRecord.threat_count  || 0,
    currentStatus: ipRecord.status        || 'NORMAL',
  });

  const model2Result = ML_MODEL_2.predict({
    threatStrikes:  ipRecord.threat_count || 0,
    model1Decision: model1Result.label,
    currentStatus:  ipRecord.status       || 'NORMAL',
  });

  const priority    = { BANNED: 3, THREAT: 2, NORMAL: 1 };
  const finalStatus = (priority[model2Result.label] || 1) >= (priority[model1Result.label] || 1)
    ? model2Result.label : model1Result.label;

  return {
    ip:           ipRecord.ip,
    model1:       { modelName: ML_MODEL_1.name, ...model1Result },
    model2:       { modelName: ML_MODEL_2.name, ...model2Result },
    finalDecision: finalStatus,
    timestamp:    new Date(now).toISOString(),
  };
}

module.exports = {
  ML_MODEL_1,
  ML_MODEL_2,
  runMLPipeline,
  MODEL_METADATA: {
    model1: {
      name:        ML_MODEL_1.name,
      algorithm:   ML_MODEL_1.algorithm,
      description: ML_MODEL_1.description,
      version:     ML_MODEL_1.version,
```

```
        hyperparams: { weights: SAVED_MODEL.weights, bias: SAVED_MODEL.bias, threshold: SAVED_MODEL.classificationThreshold, norm: SAVED_MODEL.norm },
      metrics:     SAVED_MODEL.metrics,
    },
    model2: {
      name:        ML_MODEL_2.name,
      algorithm:   ML_MODEL_2.algorithm,
      description: ML_MODEL_2.description,
      version:     ML_MODEL_2.version,
      hyperparams: ML_MODEL_2.hyperparams,
    },
  },
};
```

server/src/routes/honeypot.js

```
'use strict';

const express = require('express');
const router  = express.Router();
const db      = require('../db/database');

let lastEsp32Ping = null;

function logAttack(req, payload) {
  const ip        = req.headers['x-forwarded-for'] || req.ip || req.connection.remoteAddress || 'unknown';
  const userAgent = req.headers['user-agent'] || 'unknown';
  const status    = req.ipStatus || (() => {
    const tracker = db.prepare('SELECT status FROM ip_tracker WHERE ip = ?').get(ip);
    return tracker ? tracker.status : 'NORMAL';
  })();

  try {
    db.prepare(`
      INSERT INTO attack_log (ip, method, path, payload, user_agent, status)
      VALUES (?, ?, ?, ?, ?, ?)
    `).run(ip, req.method, req.originalUrl, payload, userAgent, status);
  } catch (err) {
    console.error('[Honeypot] DB log error:', err.message);
  }
}

// Intentionally vulnerable login endpoint — DO NOT sanitize
router.post('/login', (req, res) => {
  const { username = '', password = '' } = req.body;
  const payload = `username=${username}&password=${password}`;

  // Skip logging for ESP32-forwarded requests — /api/esp32/alert handles those
  if (!req.headers['x-forwarded-for']) {
```

```
    logAttack(req, payload);
  }

    const query = `SELECT * FROM users WHERE username='${username}' AND password='${password}'`;
  let result  = null;
  let sqlError = null;

  try {
    result = db.prepare(query).get();
  } catch (err) {
    sqlError = err.message;
    console.log(`[Honeypot] SQL Error from ${req.ip}: ${err.message}`);
  }

  if (sqlError) {
    return res.status(200).json({
      success: false,
      message: 'An internal error occurred. Please try again.',
      debug: sqlError,
    });
  }

  if (result) {
    return res.status(200).json({
      success: true,
      message: `Welcome back, ${result.username}!`,
      role: result.role,
      redirect: '/employee-portal',
    });
  }

  return res.status(200).json({ success: false, message: 'Invalid username or password.' });
});

router.get('/honeypot-status', (req, res) => {
  const count = db.prepare('SELECT COUNT(*) as total FROM attack_log').get();
  res.json({ status: 'ACTIVE', total_attacks_logged: count.total });
});

// ESP32 posts here when it detects SQLi: { attacker_ip, payload, score }
router.post('/api/esp32/alert', (req, res) => {
  const { attacker_ip, payload, score } = req.body;

  if (!attacker_ip) {
    return res.status(400).json({ error: 'attacker_ip is required' });
  }

  const ip       = attacker_ip;
  const severity = score >= 2 ? 'BANNED' : 'THREAT';
```

```
  try {
    // Update the most recent row from this IP (forwarded by ESP32) within 30s
    const updated = db.prepare(`
      UPDATE attack_log SET status = ?
      WHERE id = (
        SELECT id FROM attack_log
        WHERE ip = ? AND datetime(timestamp) >= datetime('now', '-30 seconds')
        ORDER BY id DESC LIMIT 1
      )
    `).run(severity, ip);

    if (updated.changes === 0) {
      db.prepare(`
        INSERT INTO attack_log (ip, method, path, payload, user_agent, status)
        VALUES (?, 'POST', '/login', ?, 'ESP32-PhantomIDS/1.0', ?)
      `).run(ip, payload || '', severity);
    }

    const existing = db.prepare('SELECT * FROM ip_tracker WHERE ip = ?').get(ip);
    if (existing) {
      db.prepare(`
        UPDATE ip_tracker
        SET status = CASE WHEN status != 'BANNED' THEN ? ELSE 'BANNED' END
        WHERE ip = ?
      `).run(severity, ip);
    } else {
      db.prepare(`
        INSERT INTO ip_tracker (ip, request_count, window_start, status, threat_count)
        VALUES (?, 1, datetime('now'), ?, 1)
      `).run(ip, severity);
    }

    lastEsp32Ping = Date.now();
    console.log(`[ESP32] ${score >= 2 ? 'CRITICAL' : 'WARNING'} from ${ip} — ${severity}`);
    return res.status(201).json({ status: 'ok', message: 'Alert logged', severity });
  } catch (err) {
    console.error('[ESP32] Alert ingest error:', err.message);
    return res.status(500).json({ error: err.message });
  }
});

router.post('/api/esp32/ping', (req, res) => {
  lastEsp32Ping = Date.now();
  res.json({ status: 'ok' });
});

router.get('/api/esp32/status', (req, res) => {
  const online = lastEsp32Ping && (Date.now() - lastEsp32Ping) < 3 * 60 * 1000;
  res.json({
```

```
    online: !!online,
    last_ping: lastEsp32Ping ? new Date(lastEsp32Ping).toISOString() : null,
    sensor_ip: '192.168.1.30',
  });
});

module.exports = router;
```

server/src/routes/honeypot.js

```
'use strict';

const express = require('express');
const router  = express.Router();
const db      = require('../db/database');

let lastEsp32Ping = null;

function logAttack(req, payload) {
  const ip        = req.headers['x-forwarded-for'] || req.ip || req.connection.remoteAddress || 'unknown';
  const userAgent = req.headers['user-agent'] || 'unknown';
  const status    = req.ipStatus || (() => {
    const tracker = db.prepare('SELECT status FROM ip_tracker WHERE ip = ?').get(ip);
    return tracker ? tracker.status : 'NORMAL';
  })();

  try {
    db.prepare(`
      INSERT INTO attack_log (ip, method, path, payload, user_agent, status)
      VALUES (?, ?, ?, ?, ?, ?)
    `).run(ip, req.method, req.originalUrl, payload, userAgent, status);
  } catch (err) {
    console.error('[Honeypot] DB log error:', err.message);
  }
}

// Intentionally vulnerable login endpoint — DO NOT sanitize
router.post('/login', (req, res) => {
  const { username = '', password = '' } = req.body;
  const payload = `username=${username}&password=${password}`;

  // Skip logging for ESP32-forwarded requests — /api/esp32/alert handles those
  if (!req.headers['x-forwarded-for']) {
    logAttack(req, payload);
  }

    const query = `SELECT * FROM users WHERE username='${username}' AND
password='${password}'`;
  let result  = null;
```

```
  let sqlError = null;

  try {
    result = db.prepare(query).get();
  } catch (err) {
    sqlError = err.message;
    console.log(`[Honeypot] SQL Error from ${req.ip}: ${err.message}`);
  }

  if (sqlError) {
    return res.status(200).json({
      success: false,
      message: 'An internal error occurred. Please try again.',
      debug: sqlError,
    });
  }

  if (result) {
    return res.status(200).json({
      success: true,
      message: `Welcome back, ${result.username}!`,
      role: result.role,
      redirect: '/employee-portal',
    });
  }

  return res.status(200).json({ success: false, message: 'Invalid username or password.' });
});

router.get('/honeypot-status', (req, res) => {
  const count = db.prepare('SELECT COUNT(*) as total FROM attack_log').get();
  res.json({ status: 'ACTIVE', total_attacks_logged: count.total });
});

// ESP32 posts here when it detects SQLi: { attacker_ip, payload, score }
router.post('/api/esp32/alert', (req, res) => {
  const { attacker_ip, payload, score } = req.body;

  if (!attacker_ip) {
    return res.status(400).json({ error: 'attacker_ip is required' });
  }

  const ip       = attacker_ip;
  const severity = score >= 2 ? 'BANNED' : 'THREAT';

  try {
    // Update the most recent row from this IP (forwarded by ESP32) within 30s
    const updated = db.prepare(`
      UPDATE attack_log SET status = ?
      WHERE id = (
```

```
      SELECT id FROM attack_log
      WHERE ip = ? AND datetime(timestamp) >= datetime('now', '-30 seconds')
      ORDER BY id DESC LIMIT 1
    )
  `).run(severity, ip);

  if (updated.changes === 0) {
    db.prepare(`
      INSERT INTO attack_log (ip, method, path, payload, user_agent, status)
      VALUES (?, 'POST', '/login', ?, 'ESP32-PhantomIDS/1.0', ?)
    `).run(ip, payload || '', severity);
  }

  const existing = db.prepare('SELECT * FROM ip_tracker WHERE ip = ?').get(ip);
  if (existing) {
    db.prepare(`
      UPDATE ip_tracker
      SET status = CASE WHEN status != 'BANNED' THEN ? ELSE 'BANNED' END
      WHERE ip = ?
    `).run(severity, ip);
  } else {
    db.prepare(`
      INSERT INTO ip_tracker (ip, request_count, window_start, status, threat_count)
      VALUES (?, 1, datetime('now'), ?, 1)
    `).run(ip, severity);
  }

  lastEsp32Ping = Date.now();
  console.log(`[ESP32] ${score >= 2 ? 'CRITICAL' : 'WARNING'} from ${ip} — ${severity}`);
  return res.status(201).json({ status: 'ok', message: 'Alert logged', severity });
 } catch (err) {
  console.error('[ESP32] Alert ingest error:', err.message);
  return res.status(500).json({ error: err.message });
 }
});

router.post('/api/esp32/ping', (req, res) => {
 lastEsp32Ping = Date.now();
 res.json({ status: 'ok' });
});

router.get('/api/esp32/status', (req, res) => {
 const online = lastEsp32Ping && (Date.now() - lastEsp32Ping) < 3 * 60 * 1000;
 res.json({
  online: !!online,
  last_ping: lastEsp32Ping ? new Date(lastEsp32Ping).toISOString() : null,
  sensor_ip: '192.168.1.30',
 });
});
```

```
module.exports = router;
```

server/src/db/database.js

```
'use strict';

const Database = require('better-sqlite3');
const path     = require('path');
const fs       = require('fs');

const dbDir = path.join(__dirname);
if (!fs.existsSync(dbDir)) fs.mkdirSync(dbDir, { recursive: true });

const db = new Database(path.join(__dirname, 'phantom.db'));

db.pragma('journal_mode = WAL');
db.pragma('synchronous = NORMAL');
db.pragma('cache_size = 10000');
db.pragma('temp_store = MEMORY');

// Honeypot bait users — intentionally weak credentials for sqlmap to dump
db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL,
    password TEXT NOT NULL,
    role     TEXT DEFAULT 'employee'
  )
`);

const userCount = db.prepare('SELECT COUNT(*) as count FROM users').get();
if (userCount.count === 0) {
  const insert = db.prepare('INSERT INTO users (username, password, role) VALUES (?, ?, ?)');
  insert.run('admin',      'admin123',   'admin');
  insert.run('john.doe',   'password1',  'employee');
  insert.run('jane.smith', 'letmein',    'employee');
  insert.run('root',       'toor',       'superuser');
}

// Secure admin accounts for the dashboard (bcrypt-hashed)
db.exec(`
  CREATE TABLE IF NOT EXISTS admins (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    org      TEXT NOT NULL
  )
`);
```

```
const adminCount = db.prepare('SELECT COUNT(*) as count FROM admins').get();
if (adminCount.count === 0) {
  const bcrypt = require('bcryptjs');
  const hash   = bcrypt.hashSync('PhantomAdmin@2024', 10);
  db.prepare('INSERT INTO admins (username, password, org) VALUES (?, ?, ?)').run('phantom_admin',
hash, 'PhantomIDS Security');
}

db.exec(`
  CREATE TABLE IF NOT EXISTS attack_log (
    id        INTEGER PRIMARY KEY AUTOINCREMENT,
    ip        TEXT NOT NULL,
    method    TEXT,
    path      TEXT,
    payload   TEXT,
    user_agent TEXT,
    timestamp  DATETIME DEFAULT CURRENT_TIMESTAMP,
    status    TEXT DEFAULT 'NORMAL'
  )
`);

db.exec(`
  CREATE TABLE IF NOT EXISTS ip_tracker (
    ip           TEXT PRIMARY KEY,
    request_count INTEGER DEFAULT 0,
    window_start  DATETIME DEFAULT CURRENT_TIMESTAMP,
    status        TEXT DEFAULT 'NORMAL',
    threat_count  INTEGER DEFAULT 0
  )
`);

// Safe migration for older schemas missing threat_count
try {
  db.exec('ALTER TABLE ip_tracker ADD COLUMN threat_count INTEGER DEFAULT 0');
} catch (_) {}

db.exec(`
  CREATE INDEX IF NOT EXISTS idx_attack_log_ip        ON attack_log(ip);
  CREATE INDEX IF NOT EXISTS idx_attack_log_timestamp ON attack_log(timestamp);
  CREATE INDEX IF NOT EXISTS idx_attack_log_status    ON attack_log(status);
`);

module.exports = db;
```

server/src/ml/model.json

```
{
  "algorithm": "Logistic Regression (Binary Cross-Entropy, Gradient Descent)",
  "version": "2.0.0",
```

```
  "trainedAt": "2026-04-01T16:17:13.278Z",
  "epochs": 3000,
  "learningRate": 0.05,
  "trainingSamples": 101,
  "weights": [
    4.948536,
    3.853035
  ],
  "bias": -2.044437,
  "norm": {
    "requestRate": {
      "min": 1,
      "max": 80
    },
    "threatCount": {
      "min": 0,
      "max": 3
    }
  },
  "metrics": {
    "accuracy": 86.14,
    "precision": 87.93,
    "recall": 87.93,
    "f1": 87.93
  },
  "classificationThreshold": 0.5,
  "labels": {
    "0": "NORMAL",
    "1": "THREAT"
  }
}
```

CHAPTER-6 RESULTS AND APPLICATIONS

6.1 TESTING & VALIDATION RESULTS

6.1.1 UNIT TESTING

Test Case ID	Test Case Description	Steps	Test Data	Input	Expected Result	Actual Result	Status
TC-01	Valid user login	Start honeypot → Submit valid credentials	admin / Admin@1234	POST /login	Welcome message displayed	Login successful	PASS
TC-02	SQLi bypass	Submit SQLi payload	' OR '1'='1'--	POST /login	Login bypass without password	Bypass successful	PASS
TC-03	ESP32 WiFi connection	Flash → Power ON → Open Serial	SSID: Fibercity	WiFi.begin()	IP printed on connection	Connected, IP shown	PASS
TC-04	ESP32 SQLi detection	Boot ESP32 → Send SQLi → Observe	SQLi payload	curl POST	Alert: LCD, red LED, buzzer	Alert triggered correctly	PASS
TC-05	Clean request handling	Send valid login → Observe	Valid credentials	curl POST	No alert, CLEAN state	No alert triggered	PASS
TC-06	URL decoding	Send encoded payload → Check output	Encoded SQLi	POST request	Correct decoding, alert	Decoded & detected	PASS
TC-07	Attack logging	Send SQLi → Check logs	IP: 192.168.1.7	POST /login	Log entry created	Entry recorded	PASS

TC-08	sqlmap dump	Run sqlmap command	users table	sqlmap command	All users extracted	Data dumped correctly	PASS

Table 6.1.1 : Unit Test Cases

6.1.2 SYSTEM TESTING

Test Case ID	Test Case Description	Steps	Test Data	Input	Expected Result	Actual Result	Status
ST-01	End-to-end SQLi detection	1. Start server 2. Power ESP32 3. Send SQLi 4. Check dashboard	Payload: ' OR '1'='1'-- IP: 192.168.1.7	POST to ESP32	Detection, alert, ML = THREAT, logged, dashboard update	All components worked correctly	PASS
ST-02	3-strike auto-ban	1. Start server 2. Send 3 SQLi 3. Check IP status	IP: 192.168.1.100	3 malicious requests	IP banned after 3rd attempt, further blocked	IP banned successfully	PASS
ST-03	Dashboard real-time update	1. Login 2. Trigger attack 3. Observe	Admin creds + SQLi	Browser + POST	Dashboard shows new attack entry	Updated within few seconds	PASS
ST-04	ESP32 proxy working	1. Boot ESP32 2. Send valid login	admin/admin123	POST via ESP32	Request forwarded, valid response returned	Login successful	PASS

ST-05	ML model performance	1. Run simulation 2. Generate traffic	50 normal + 50 attack	npm run simulate	Accuracy ≥85%	Accuracy ~86%, F1 ~88%	PASS
ST-06	Admin authentication	1. Access without login 2. Login 3. Recheck	Valid/Invalid creds	Browser login	Secure login, session handling	Auth worked correctly	PASS
ST-07	Database persistence	1. Send multiple attacks 2. Check DB	Multiple IPs	Script/tool	All logs stored, no data loss	Records saved correctly	PASS
ST-08	sqlmap attack handling	1. Run sqlmap 2. Monitor system	sqlmap scan	sqlmap command	Alerts, logs, dashboard updates	Multiple attacks logged	PASS

Table 6.1.2 : System Test Cases

6.2 REAL-WORLD APPLICATIONS

The PhantomIDS framework offers several practical applications in modern cybersecurity, particularly as a specialized tool for detecting application-layer threats that traditional perimeter firewalls may miss.

Internal Network Auditing and Compliance Organizations in highly regulated sectors, such as banking and healthcare, are required by standards like PCI DSS 4.0 and HIPAA to maintain deep visibility into application-layer traffic. PhantomIDS can be deployed as an internal auditing sensor to monitor traffic within local subnets. Because it operates as a transparent HTTP proxy, it provides real-time inspection of requests without requiring any modifications to the production web servers. This physical isolation ensures that security logs remain tamper-proof even if an attacker gains shell access to the target host, satisfying the core IDS requirements defined by NIST SP 800-94.

Threat Intelligence and Honeypot Traps A critical application of this system is its use as a forensic data collection tool. By mimicking a legitimate corporate portal, the integrated honeypot lures automated attack tools and bad bots into a controlled environment. This setup allows security researchers to capture the characteristic sequential probe patterns used by tools like sqlmap. The behavioral data collected—such as request rates and payload entropy—is used to train machine learning models that can identify zero-day SQLi variants and obfuscated payloads, which are then used to inform the rule sets of enterprise-grade solutions like AWS WAF or Google Cloud Armor.

Out-of-Band Alerting for IoT and OT Environments As Industrial IoT (IIoT) and Operational Technology (OT) processes migrate to cloud-based management, they become vulnerable to novel WAF bypass techniques. PhantomIDS provides a resilient defense for these environments because the ESP32 hardware is physically separate from the systems it protects, making it unreachable via software-based "kill" commands. Its ability to fire immediate physical alerts via an LCD display and buzzer offers an "out-of-band" security heartbeat for operators, ensuring that network intrusions are noticed even if digital management dashboards are compromised or bypassed using JSON-based evasion techniques.

7 CONCLUSION AND REFERENCES

CONCLUSION

PhantomIDS successfully demonstrates a physically isolated, hardware-based intrusion detection system that overcomes the critical vulnerability of traditional host-resident software WAFs, which can be disabled if an attacker gains shell access. By deploying an ESP32 microcontroller as a transparent HTTP proxy within a WiFi broadcast domain, the system achieves real-time inspection of traffic destined for a deliberately vulnerable honeypot. This architecture utilizes a two-signal confidence scoring mechanism—cross-referencing URL-decoded SQL keywords against suspicious character presence—to trigger immediate physical alerts via a 16x2 LCD and active buzzer, ensuring that security monitoring remains independent and unreachable by software-level attacks.

Beyond physical alerting, the project integrates a sophisticated machine learning pipeline that elevates its detection capabilities from static signature matching to behavioral anomaly analysis. With a logistic regression model currently boasting 86.14% accuracy, the backend effectively correlates request rates and threat counts to automate network defense through a deterministic 3-strike auto-ban engine. This hybrid approach not only satisfies the practical requirements of the MSBTE syllabus (Units 5.1 and 5.4) but also offers a research-grade platform for capturing and analyzing the characteristic sequential probe patterns of automated attack tools like sqlmap.

REFERENCES

[1] A. Sameh and S. Selim, "Adaptive Dual-Layer Web Application Firewall (ADL-WAF) Leveraging Machine Learning for Enhanced Anomaly and Threat Detection," arXiv, 2024.

[2] N. C. Nandigama, "Predictive SQL Injection Detection and Prevention Using Machine Learning Across AWS, Azure, and Google Cloud Platforms," International Journal of Engineering Science and Advanced Technology (IJESAT), 2024.

[3] Amazon Web Services, "Security Automations for AWS WAF: Implementation Guide," AWS Documentation, 2026.

[4] Microsoft, "CRS and DRS Rule Groups and Rules - Azure Web Application Firewall," Microsoft Learn, 2026.

[5] NSFOCUS, "Web Application Firewall (WAF) Datasheet: Advanced, Innovative Technology," NSFOCUS Global, 2024.

[6] A. Ribeiro, "Claroty's Team82 Develops Generic Bypass of WAF, Calls for Review of JSON Support Across Organizations," Industrial Cyber, 2022.

[7] S. Yeluri, "The Evolution of Network Security," APNIC Blog, 2024.

[8] A. Patel, "AWS WAF vs. Google Cloud Armor: A Multicloud Security Showdown," The New Stack, 2025.